1. CONCEPTUAL MODEL OF UML

To understand UML, you need to form a conceptual model of the language. This requires learning three major elements:

I. Basic Building Blocks

II. Rules

III. Common Mechanisms

I) BASIC BUILDING BLOCKS

There are 3 kinds of basic building blocks

1. Things
2. Relationships
3. Diagrams

1) *THINGS* in the UML

There are 4 kinds of things in the UML

i. Structural Things

ii. Behavioral Things

iii. Annotational Things

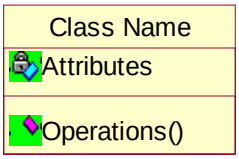
iv. Grouping Things

*1)***Structural Things**

Structural Things are nouns of UML models. There are 7 kinds of Structural things

a) C*lass*

A class is a set of objects that share the same attributes, operations, relationships and semantics.



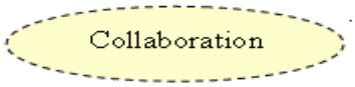
b) *Interface*

An Interface is a collection of operations that specify a service of a class or component



c) *Collaboration*

Collaboration describes co-operative work of an element.



d) *Use Case*

Use Case describes set of sequence of actions that a system performs that yields an observable result of value to a particular actor.



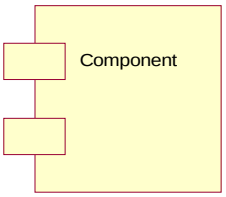
e) ***Active Class***

Active class is just like a class but its elements are connected to other class elements.



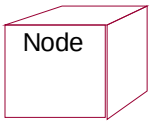
f) ***Component***

Component represents physical packaging of logical elements like classes, interfaces and collaborations.



g) ***Node***

A Node is physical element that exists at run-time and having at least some memory and processing capability.



ii) ***Behavioral Things***

Behavioral Things are the verbs of UML representing behavior over time and space. There are 2 kinds of Behavioral things

a) ***Interaction***

Interaction is used to show communication between two objects.



b) **S*tate Machine***

State Machine specifies sequence of states of an object.

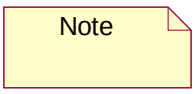


iii) ***Annotational Things***

Annotational Things are explanatory parts of UML. Only 1 type.

Note:

Note is used to give comments to an element or collection of elements.

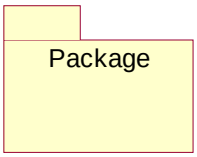


iv) ***Grouping Things***

Grouping Things are the organizational parts of the model. Only one type.

Package

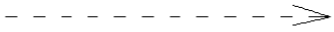
Package is a general purpose mechanism for organizing elements or things into groups or packages.



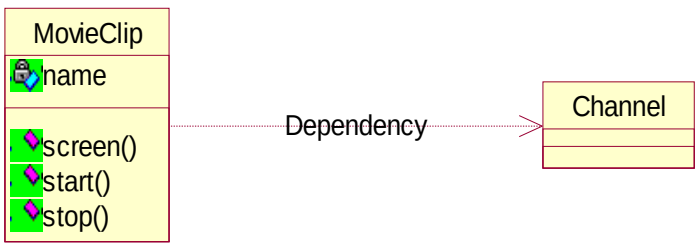
2) ***RELATIONSHIPS***

There are 4 kinds of Relationships in the UML:

a) ***Dependency*****:** It is denoted by dashed line with an arrow.

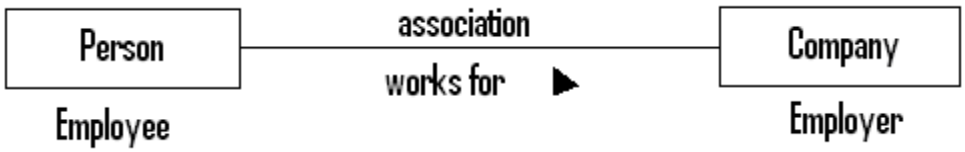


Dependency is a relationship between two things in which a change to one thing (Independent thing) may affect the other thing (Dependent thing).

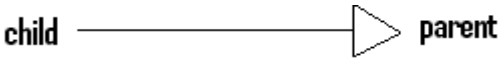


b) ***Association*:** It is denoted by a solid line.

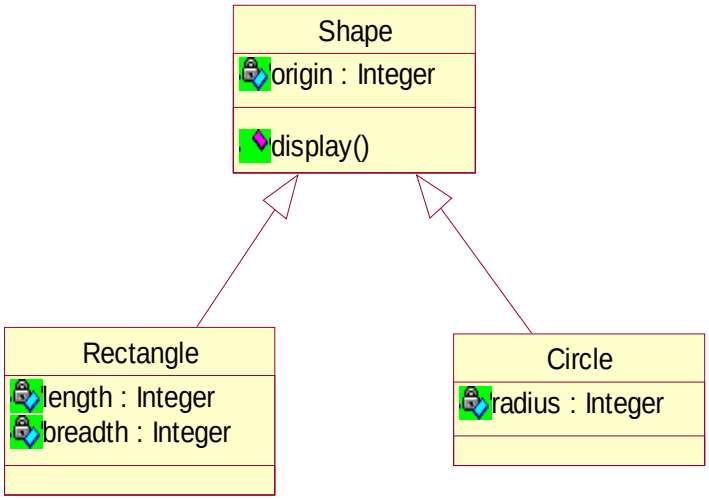
Association is a structural relationship that describes a set of links, a link being a connection among objects. Aggregation is a special kind of association, representing a structural relationship between a whole and its parts.



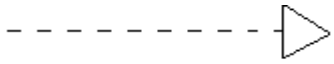
c) ***Generalization*:** It is denoted by a solid line with a hollow arrow head pointing to the parent



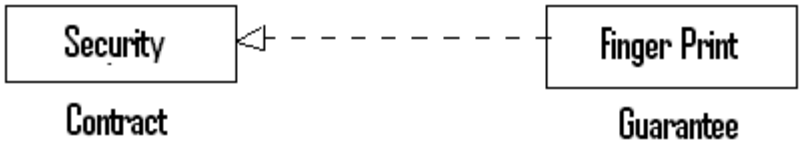
Generalization is a relationship in which the child will share the behavior of the parent.



d) ***Realization*:** It is denoted by dashed lines with a hollow arrow head.



Realization is a relationship between classifiers, where one classifier specifies a contract that another classifier guarantees to carry out



3) ***DIAGRAMS***

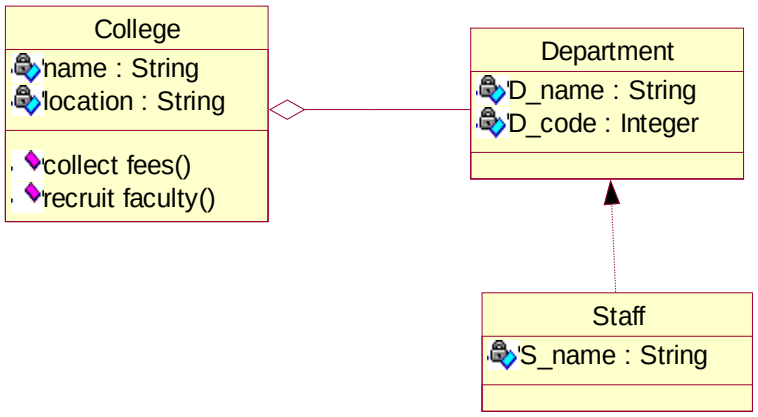
There are 9 types of Diagrams in UML, which are classified into 2 types

i) ***Structural Diagrams*** *(static diagrams)*

These are of 4 types

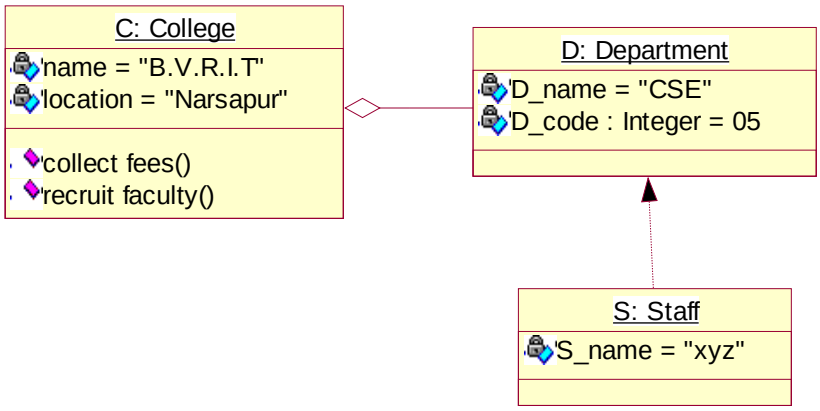
a) Class Diagram

A Class diagram shows a set of classes, interfaces, and collaborations and their relationships. A class consists of class name, attributes, operations and responsibilities.



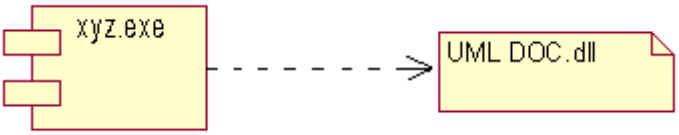
b) ***Object Diagram***

An Object diagram shows a set of objects and their relationships. They represent snapshots of instances of the things found in class diagrams.



c) ***Component Diagram***

A Component diagram shows the organizations and dependencies among a set of components.



d) ***Deployment Diagram***

A Deployment diagram shows the configuration of run-time processing nodes and the components that are present in them.



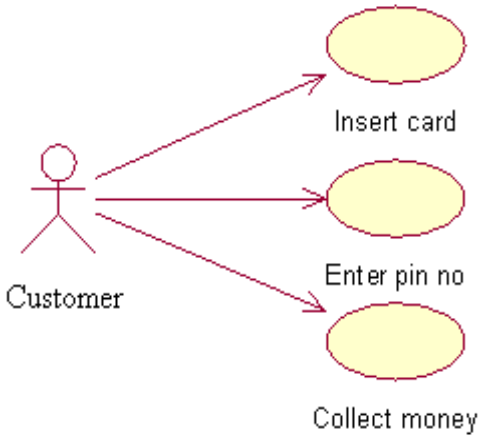
Component and Deployment diagrams are called as Physical Diagrams.

ii) **Behavioral *Diagrams***(Dynamic diagrams)

These are of 5 types

a) ***Use Case Diagram***

A Use Case diagram shows a set of use cases and actors and their relationships. An ***Actor*** can be a human or a system. The role of actor is written below.



Interaction Diagrams

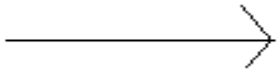
An Interaction diagram shows an interaction, consisting of a set of objects and their relationships, including the messages they exchange among them.

2 types of Interaction diagrams are:

b) ***Sequence Diagram***

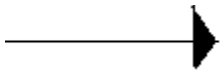
A Sequence diagram is an interaction diagram that emphasizes the time-ordering of messages. To show interaction between objects we use 3 types of messages.

***Simple Messages*:**



A Simple message shows how control is passed from one object to other without describing communication in detail i.e. without indicating whether it is synchronous or asynchronous message.

***Synchronous Messages*:**



If sender object waits for a reply from receiver object from destination, such messages are called Synchronous messages. Here, only one object can send a message at a given instance of time.

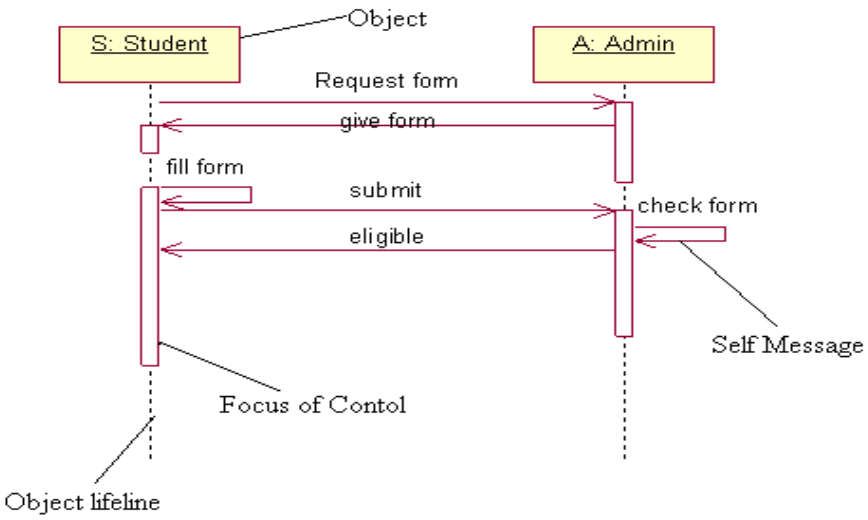
***Asynchronous Messages*:**



If sender object continues executing while target is processing the message then such messages are said to be Asynchronous messages. Here, multiple messages are executed at a time.

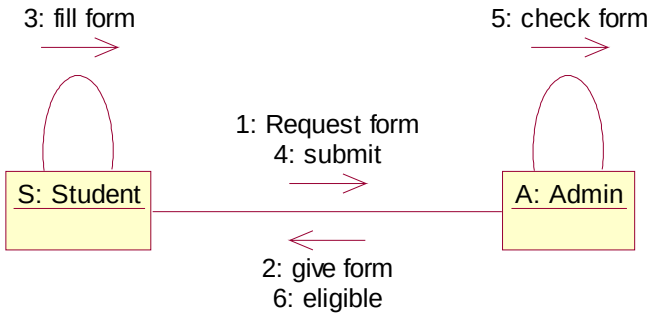
***Object Lifeline*:** An Object life line is vertical dashed lines that represent the existence of an object over a period of time.

***Focus of Control*:** It is represented by rectangle that shows the period of time during which an object performs some actions.



c) ***Collaboration Diagram***

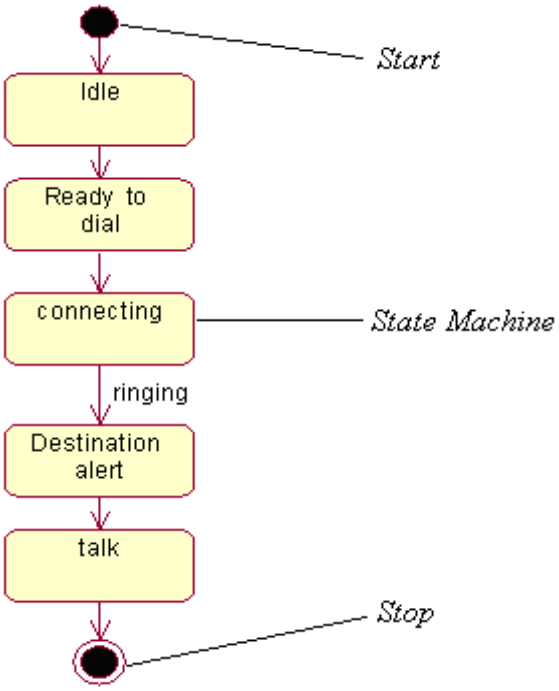
A Collaboration diagram is an interaction diagram that emphasizes the structural organization of the objects that send and receive messages.



d) ***State chart Diagram***

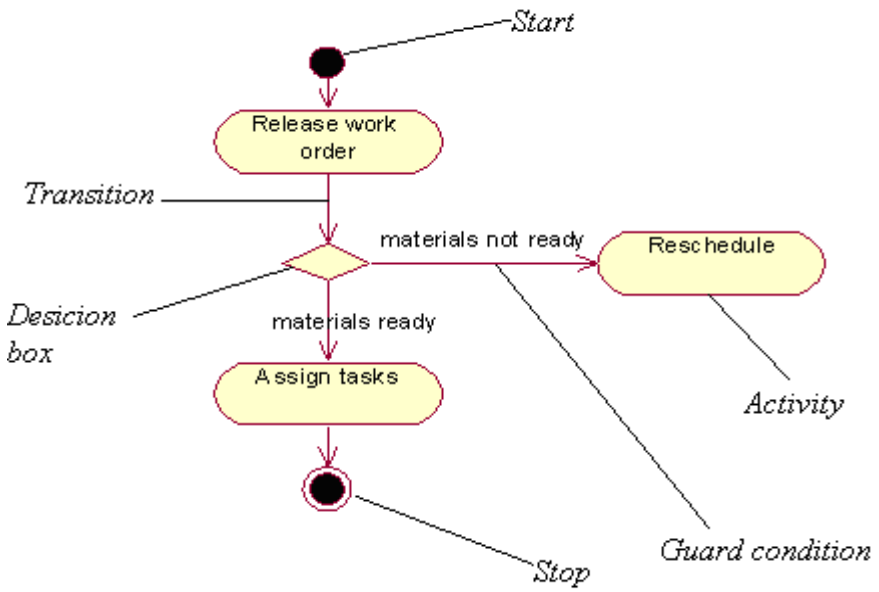
A State chart diagram shows a state machine, consisting of states, transitions, events, and activities.

***Event*:** It refers to happening of an activity at a given time and place.



e) ***Activity Diagram***

An activity diagram is a special kind of state chart diagram that shows the flow from activity to activity within a system, which are connected by a trigger less transition. We can check some conditions using decision box, which is denoted by a diamond.



Activity: It is a major task that must take place in order to fulfill an operation contract.

Initial Activity: This shows the starting point of the flow. It is denoted by solid circle

Final Activity: This shows the end of the flow in the activity diagram. It is denoted by a solid circle nested in a circle.

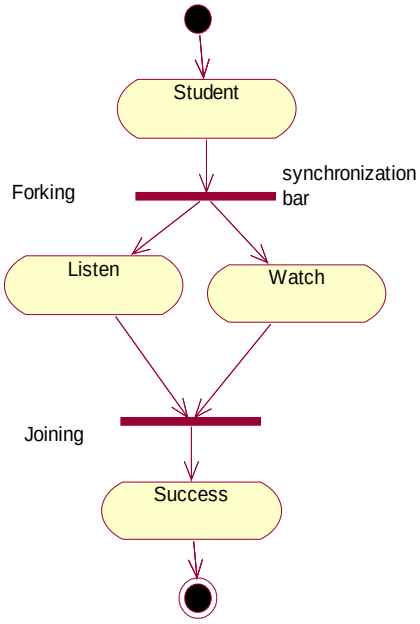
Decision Box: A point in an Activity diagram where a flow splits into several mutually exclusive guarded flows. It has one incoming transition and two outgoing transitions.

***Forking and Joining*:**-We use synchronization bar to specify the forking and joining of parallel flows of control.

A ***synchronization bar*** is a thick horizontal or vertical line.

A ***Fork*** may have one incoming transition and two or more outgoing transitions, each of which represents an independent flow of control.

A ***Join*** may have two or more incoming transitions and one outgoing transition.



***Swim lanes*:**

They are used to group related activities into one column.

UML- Architecture

Software architecture is all about how a software system is built at its highest level. It is needed to think big from multiple perspectives with quality and design in mind. The software team is tied to many practical concerns, such as:

- The structure of the development team.
- The needs of the business.
- Development cycle.
- The intent of the structure itself.

Software architecture provides a basic design of a complete software system. It defines the elements included in the system, the functions each element has, and how each element relates to one another. In short, it is a big picture or overall structure of the whole system, how everything works together.

To form an architecture, the software architect will take several factors into consideration:

- What will the system be used for?
- Who will be using the system?
- What quality matters to them?
- Where will the system run?

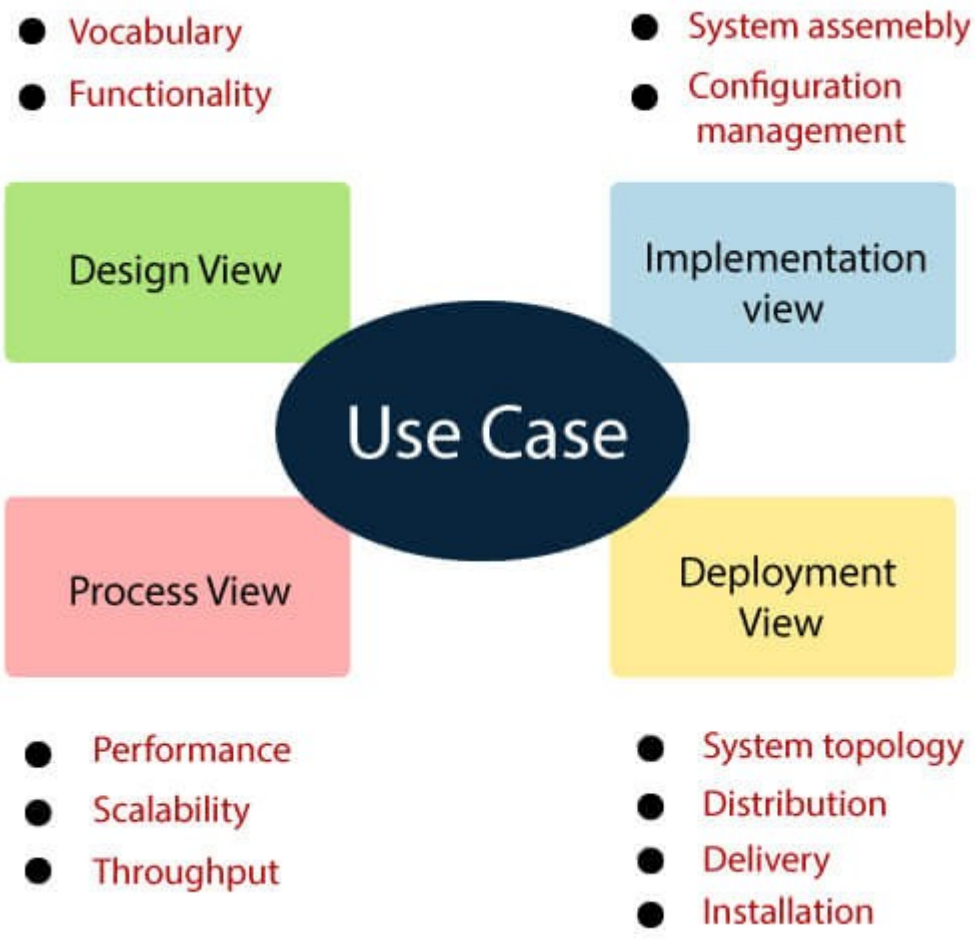
The architect plans the structure of the system to meet the needs like these. It is essential to have proper software architecture, mainly for a large software system. Having a clear design of a complete system as a starting point provides a solid basis for developers to follow.

Each developer will know what needs to be implemented and how things relate to meet the desired needs efficiently. One of the main advantages of software architecture is that it provides high productivity to the software team. The software development becomes more effective as it comes up with an explained structure in place to coordinate work, implement individual features, or ground discussions on potential issues. With a lucid architecture, it is easier to know where the key responsibilities are residing in the system and where to make changes to add new requirements or simply fixing the failures.

In addition, a clear architecture will help to achieve quality in the software with a well-designed structure using principles like separation of concerns; the system

becomes easier to maintain, reuse, and adapt. The software architecture is useful to people such as software developers, the project manager, the client, and the end-user. Each one will have different perspectives to view the system and will bring different agendas to a project. Also, it provides a collection of several views. It can be best understood as a collection of five views:

1. Use case view
2. Design view
3. Implementation view
4. Process view
5. Development view



Use case view

- It is a view that shows the functionality of the system as perceived by external actors.
- It reveals the requirements of the system.

- With UML, it is easy to capture the static aspects of this view in the use case diagrams, whereas it?s dynamic aspects are captured in interaction diagrams, state chart diagrams, and activity diagrams.

Design View

- It is a view that shows how the functionality is designed inside the system in terms of static structure and dynamic behavior.
- It captures the vocabulary of the problem space and solution space.
- With UML, it represents the static aspects of this view in class and object diagrams, whereas its dynamic aspects are captured in interaction diagrams, state chart diagrams, and activity diagrams.

Implementation View

- It is the view that represents the organization of the core components and files.
- It primarily addresses the configuration management of the system?s releases.
- With UML, its static aspects are expressed in component diagrams, and the dynamic aspects are captured in interaction diagrams, state chart diagrams, and activity diagrams.

Process View

- It is the view that demonstrates the concurrency of the system.
- It incorporates the threads and processes that make concurrent system and synchronized mechanisms.
- It primarily addresses the system's scalability, throughput, and performance.
- Its static and dynamic aspects are expressed the same way as the design view but focus more on the active classes that represent these threads and processes.

Deployment View

- It is the view that shows the deployment of the system in terms of physical architecture.

- It includes the nodes, which form the system hardware topology where the system will be executed.
- It primarily addresses the distribution, delivery, and installation of the parts that build the physical system.

Software Development Life Cycle (SDLC)

A software life cycle model (also termed process model) is a pictorial and diagrammatic representation of the software life cycle. A life cycle model represents all the methods required to make a software product transit through its life cycle stages. It also captures the structure in which these methods are to be undertaken.

In other words, a life cycle model maps the various activities performed on a software product from its inception to retirement. Different life cycle models may plan the necessary development activities to phases in different ways. Thus, no element which life cycle model is followed, the essential activities are contained in all life cycle models though the action may be carried out in distinct orders in different life cycle models. During any life cycle stage, more than one activity may also be carried out.

Need of SDLC

The development team must determine a suitable life cycle model for a particular plan and then observe to it.

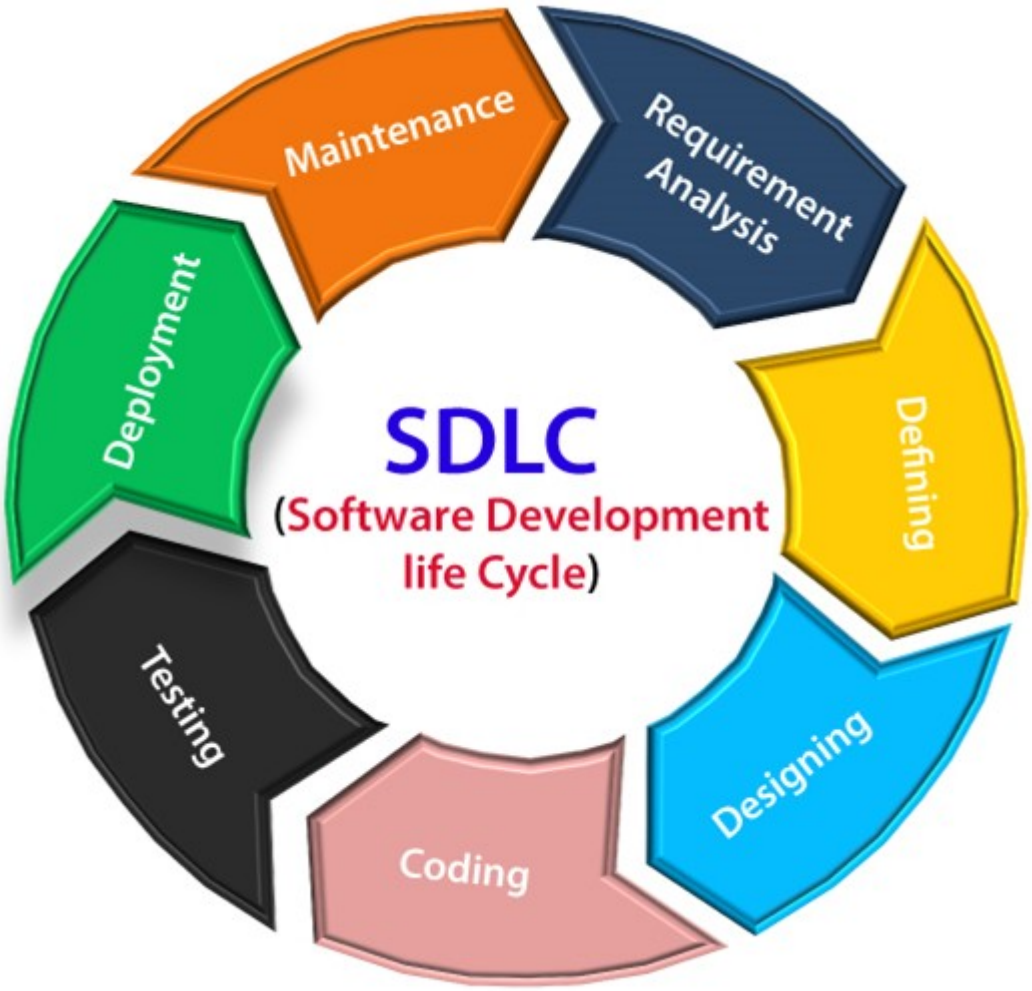
Without using an exact life cycle model, the development of a software product would not be in a systematic and disciplined manner. When a team is developing a software product, there must be a clear understanding among team representative about when and what to do. Otherwise, it would point to chaos and project failure. This problem can be defined by using an example. Suppose a software development issue is divided into various parts and the parts are assigned to the team members. From then on, suppose the team representative is allowed the freedom to develop the roles assigned to them in whatever way they like. It is possible that one representative might start writing the code for his part, another might choose to prepare the test documents first, and some other engineer might begin with the design phase of the roles assigned to him. This would be one of the perfect methods for project failure.

A software life cycle model describes entry and exit criteria for each phase. A phase can begin only if its stage-entry criteria have been fulfilled. So without a

software life cycle model, the entry and exit criteria for a stage cannot be recognized. Without software life cycle models, it becomes tough for software project managers to monitor the progress of the project.

SDLC Cycle

SDLC Cycle represents the process of developing software. SDLC framework includes the following steps:



The stages of SDLC are as follows:

Stage1: Planning and requirement analysis

Requirement Analysis is the most important and necessary stage in SDLC.

The senior members of the team perform it with inputs from all the stakeholders and domain experts or SMEs in the industry.

Planning for the quality assurance requirements and identifications of the risks associated with the projects is also done at this stage.

Business analyst and Project organizer set up a meeting with the client to gather all the data like what the customer wants to build, who will be the end user, what is the objective of the product. Before creating a product, a core understanding or knowledge of the product is very necessary.

For Example, A client wants to have an application which concerns money transactions. In this method, the requirement has to be precise like what kind of operations will be done, how it will be done, in which currency it will be done, etc.

Once the required function is done, an analysis is complete with auditing the feasibility of the growth of a product. In case of any ambiguity, a signal is set up for further discussion.

Once the requirement is understood, the SRS (Software Requirement Specification) document is created. The developers should thoroughly follow this document and also should be reviewed by the customer for future reference.

Stage2: Defining Requirements

Once the requirement analysis is done, the next stage is to certainly represent and document the software requirements and get them accepted from the project stakeholders.

This is accomplished through "SRS"- Software Requirement Specification document which contains all the product requirements to be constructed and developed during the project life cycle.

Stage3: Designing the Software

The next phase is about to bring down all the knowledge of requirements, analysis, and design of the software project. This phase is the product of the last two, like inputs from the customer and requirement gathering.

Stage4: Developing the project

In this phase of SDLC, the actual development begins, and the programming is built. The implementation of design begins concerning writing code. Developers have to follow the coding guidelines described by their management and programming tools like compilers, interpreters, debuggers, etc. are used to develop and implement the code.

Stage5: Testing

After the code is generated, it is tested against the requirements to make sure that the products are solving the needs addressed and gathered during the requirements stage.

During this stage, unit testing, integration testing, system testing, acceptance testing are done.

Stage6: Deployment

Once the software is certified, and no bugs or errors are stated, then it is deployed.

Then based on the assessment, the software may be released as it is or with suggested enhancement in the object segment.

After the software is deployed, then its maintenance begins.

Stage7: Maintenance

Once when the client starts using the developed systems, then the real issues come up and requirements to be solved from time to time.

This procedure where the care is taken for the developed product is known as maintenance.

Use-case diagrams

In UML, use-case diagrams model the behaviour of a system and help to capture the requirements of the system.

Use-case diagrams describe the high-level functions and scope of a system. These diagrams also identify the interactions between the system and its actors. The use cases and actors in use-case diagrams describe what the system does and how the actors use it, but not how the system operates internally.

Use-case diagrams illustrate and define the context and requirements of either an entire system or the important parts of the system. You can model a complex system with a single use-case diagram, or create many use-case diagrams to model the components of the system. You would typically develop use-case diagrams in the early phases of a project and refer to them throughout the development process.

Use-case diagrams are helpful in the following situations:

- Before starting a project, you can create use-case diagrams to model a business so that all participants in the project share an understanding of the workers, customers, and activities of the business.
- While gathering requirements, you can create use-case diagrams to capture the system requirements and to present to others what the system should do.
- During the analysis and design phases, you can use the use cases and actors from your use-case diagrams to identify the classes that the system requires.
- During the testing phase, you can use use-case diagrams to identify tests for the system.

The following topics describe model elements in use-case diagrams:

- **Use cases**
  A use case describes a function that a system performs to achieve the user's goal. A use case must yield an observable result that is of value to the user of the system.
- **Actors**
  An actor represents a role of a user that interacts with the system that you are modeling. The user can be a human user, an organization, a machine, or another external system.
- **Subsystems**
  In UML models, subsystems are a type of stereotyped component that represent independent, behavioral units in a system. Subsystems are used in class, component, and use-case diagrams to represent large-scale components in the system that you are modeling.
- **Relationships in use-case diagrams**
  In UML, a relationship is a connection between model elements. A UML relationship is a type of model element that adds semantics to a model by defining the structure and behaviour between the model elements.

What is a use case diagram?

In the Unified Modeling Language (UML), a use case diagram can summarize the details of your system's users (also known as actors) and their interactions with the system. To build one, you'll use a set of specialized symbols and connectors. An effective use case diagram can help your team discuss and represent:

- Scenarios in which your system or application interacts with people, organizations, or external systems
- Goals that your system or application helps those entities (known as actors) achieve
- The scope of your system

When to apply use case diagrams

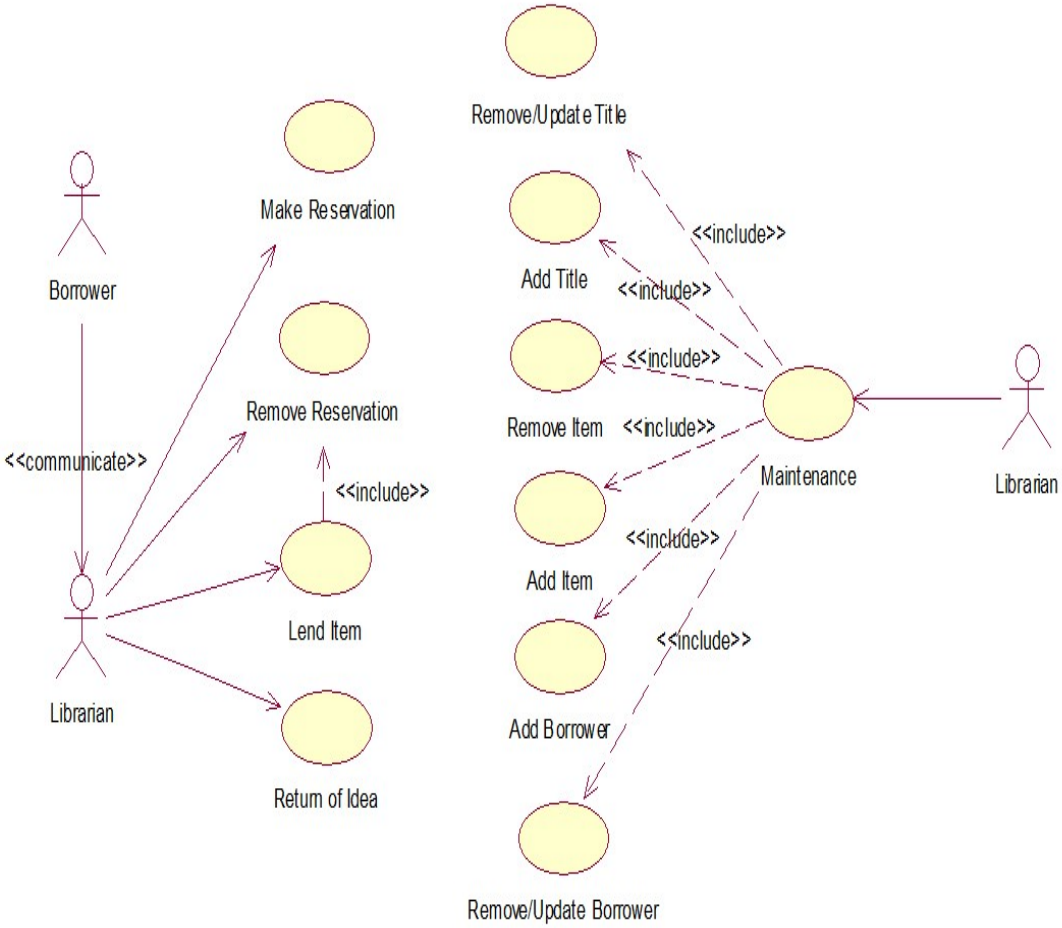
A use case diagram doesn't go into a lot of detail—for example, don't expect it to model the order in which steps are performed. Instead, a proper use case diagram depicts a high-level overview of the relationship between use cases, actors, and systems. Experts recommend that use case diagrams be used to supplement a more descriptive textual use case.

UML is the modelling toolkit that you can use to build your diagrams. Use cases are represented with a labelled oval shape. Stick figures represent actors in the process, and the actor's participation in the system is modelled with a line between the actor and use case. To depict the system boundary, draw a box around the use case itself.

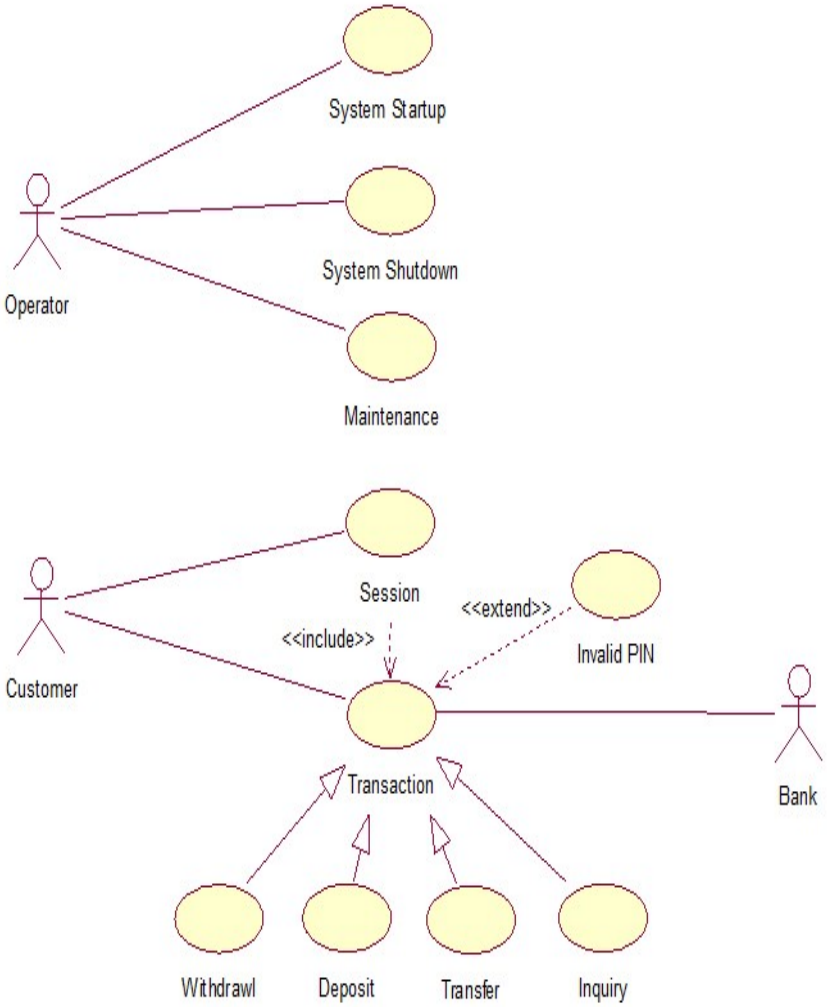
UML use case diagrams are ideal for:

- Representing the goals of system-user interactions
- Defining and organizing functional requirements in a system
- Specifying the context and requirements of a system
- Modelling the basic flow of events in a use case

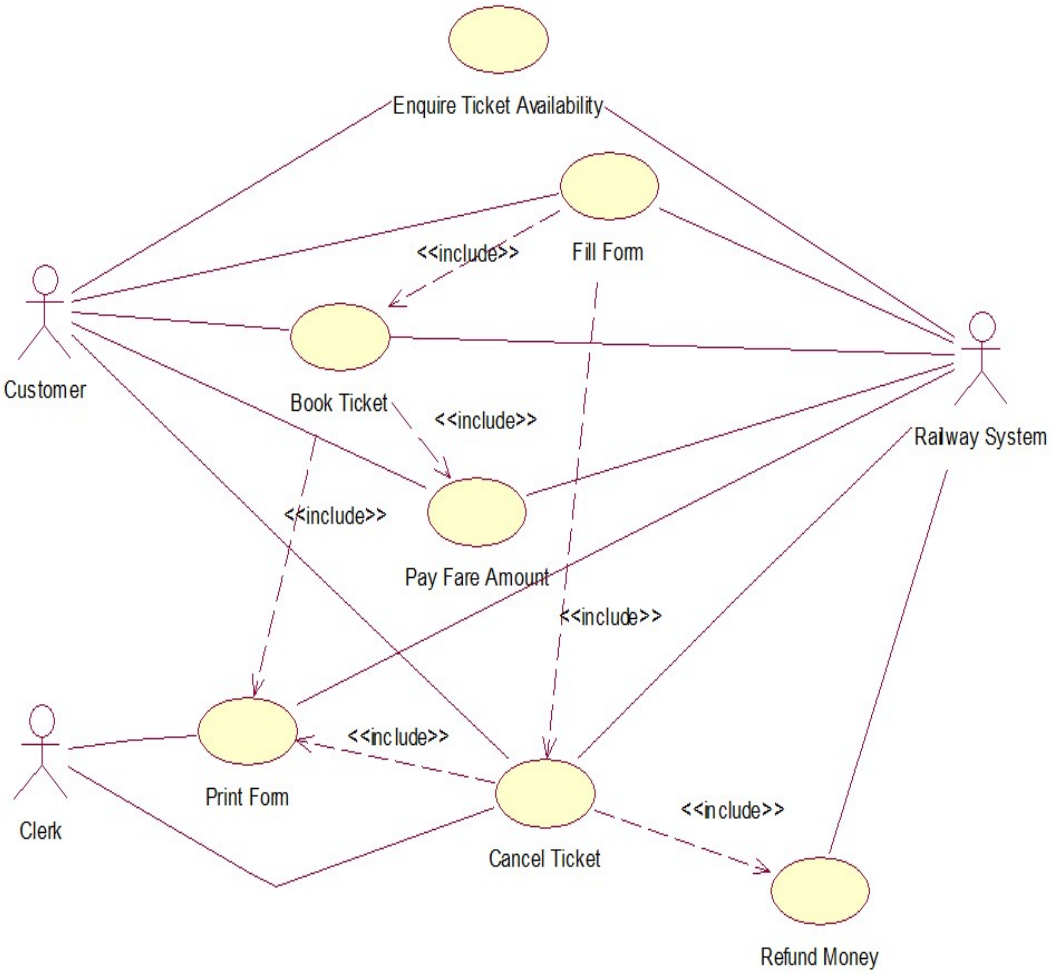
Library Management Systems


Remove/Update Title
Make Reservation
Borrower
Add Title
<<include>>
<<include>>
Remove Reservation
<<include>>
Remove Item
<<include>>
<<communicate>>
<<include>>
Maintenance
Librarian
Add Item
<<include>>
Lend Item
<<include>>
Librarian
Add Borrower
Return of Idea
Remove/Update Borrower

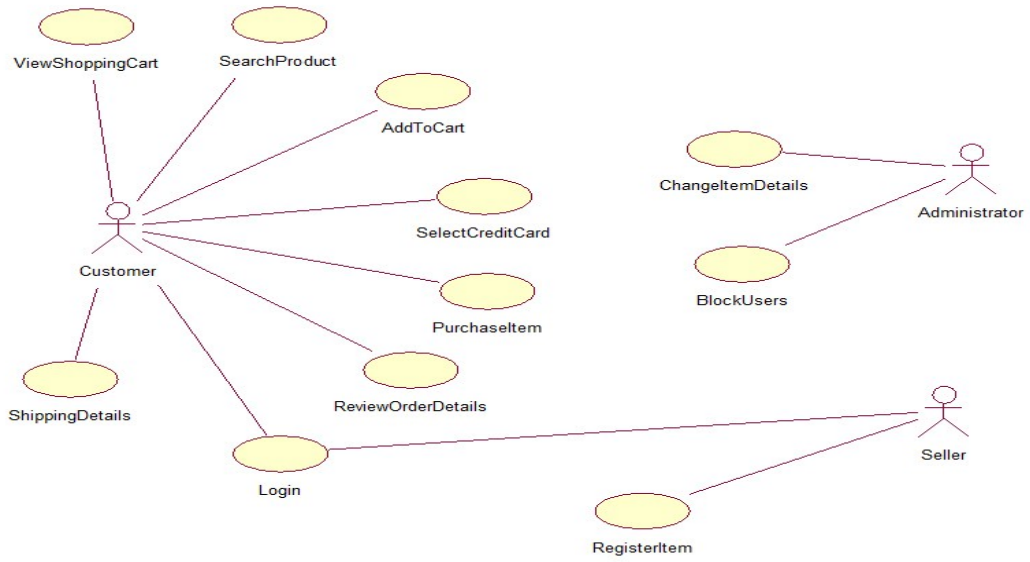
ATM



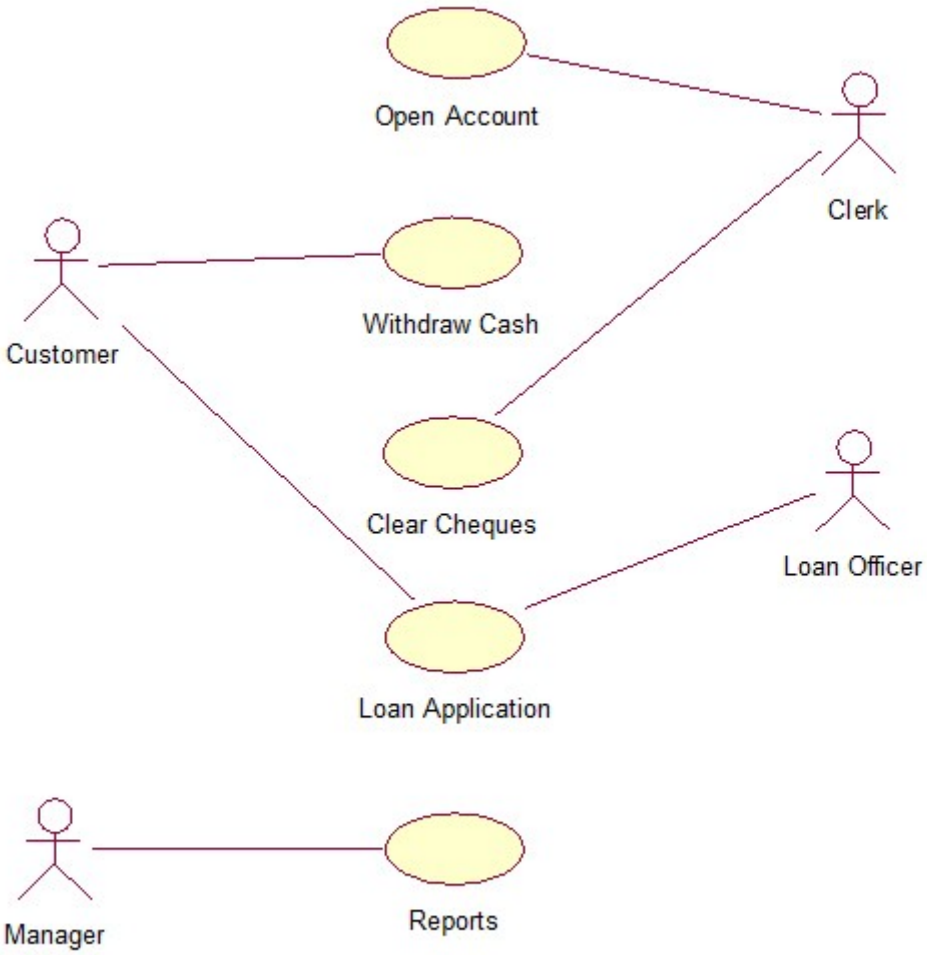
Railway Reservation



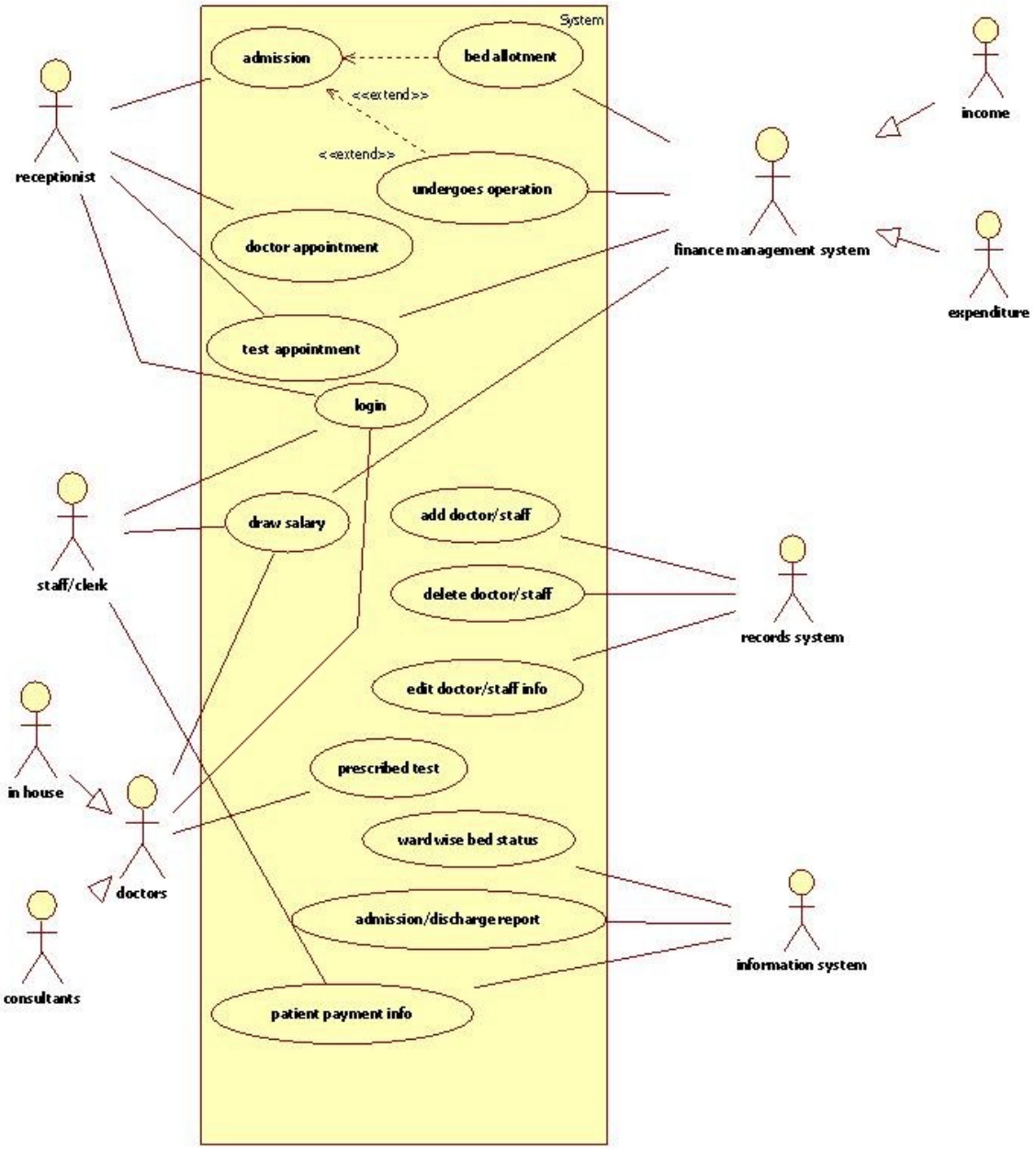
Online Book Shopping



Online Banking



Hospital Management systems


System
admission
bed allotment
<<extend>>
<<extend>>
undergoes operation
doctor appointment
test appointment
login
draw salary
add doctor/staff
delete doctor/staff
edit doctor/staff info
prescribed test
ward wise bed status
admission/discharge report
patient payment info
receptionist
staff/clerk
in house
doctors
consultants
income
expenditure
finance management system
records system
information system

Sequence Diagram

A **Sequence Diagram** simply depicts interaction between objects in a sequential order. The purpose of a sequence diagram in UML is to visualize the sequence of a message flow in the system. The sequence diagram shows the interaction between two lifelines as a time-ordered sequence of events.

- A sequence diagram shows an implementation of a scenario in the system. Lifelines in the system take part during the execution of a system.
- In a sequence diagram, a lifeline is represented by a vertical bar.
- A message flow between two or more objects is represented using a vertical dotted line which extends across the bottom of the page.
- In a sequence diagram, different types of messages and operators are used which are described above.
- In a sequence diagram, iteration and branching are also used.

Benefits of a Sequence Diagram

- Sequence diagrams are used to explore any real application or a system.
- Sequence diagrams are used to represent message flow from one object to another object.
- Sequence diagrams are easier to maintain.
- Sequence diagrams are easier to generate.
- Sequence diagrams can be easily updated according to the changes within a system.
- Sequence diagram allows reverse as well as forward engineering.

Drawbacks of a sequence diagram

- Sequence diagrams can become complex when too many lifelines are involved in the system.
- If the order of message sequence is changed, then incorrect results are produced.
- Each sequence needs to be represented using different message notation, which can be a little complex.
- The type of message decides the type of sequence inside the diagram.

The sequence diagram represents the flow of messages in the system and is also termed as an event diagram. It helps in envisioning several dynamic scenarios. It portrays the communication between any two lifelines as a time-ordered sequence of events, such that these lifelines took part at the run time. In UML, the lifeline is represented by a vertical bar, whereas the message flow is represented by a vertical dotted line that extends across the bottom of the page. It incorporates the iterations as well as branching.

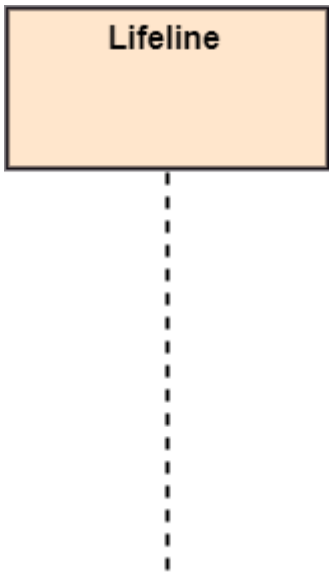
Purpose of a Sequence Diagram

1. To model high-level interaction among active objects within a system.
2. To model interaction among objects inside a collaboration realizing a use case.
3. It either models generic interactions or some certain instances of interaction.

Notations of a Sequence Diagram

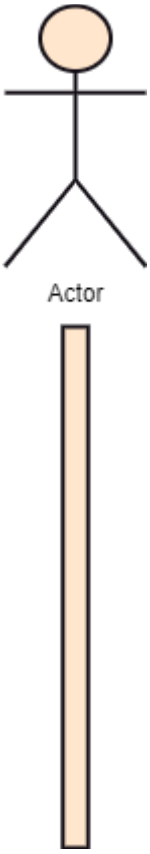
Lifeline

An individual participant in the sequence diagram is represented by a lifeline. It is positioned at the top of the diagram.



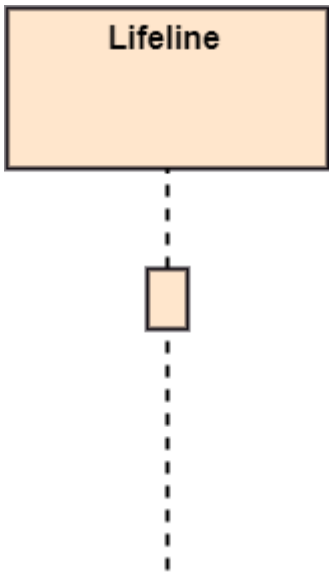
Actor

A role played by an entity that interacts with the subject is called as an actor. It is out of the scope of the system. It represents the role, which involves human users and external hardware or subjects. An actor may or may not represent a physical entity, but it purely depicts the role of an entity. Several distinct roles can be played by an actor or vice versa.



Activation

It is represented by a thin rectangle on the lifeline. It describes that time period in which an operation is performed by an element, such that the top and the bottom of the rectangle is associated with the initiation and the completion time, each respectively.

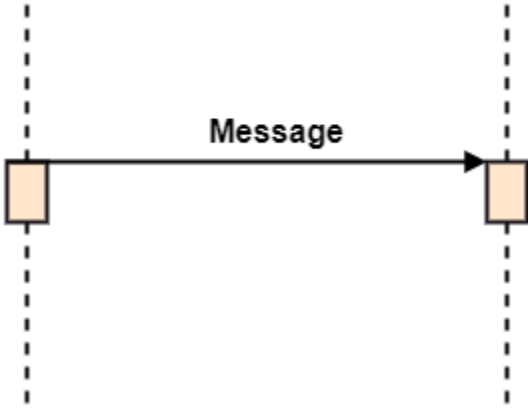


Messages

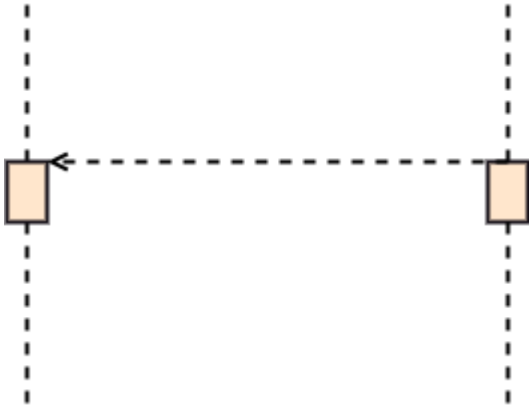
The messages depict the interaction between the objects and are represented by arrows. They are in the sequential order on the lifeline. The core of the sequence diagram is formed by messages and lifelines.

Following are types of messages enlisted below:

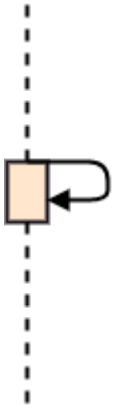
- **Call Message:** It defines a particular communication between the lifelines of an interaction, which represents that the target lifeline has invoked an operation.



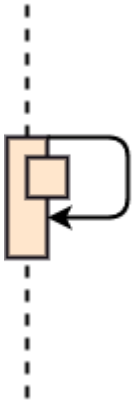
- **Return Message:** It defines a particular communication between the lifelines of interaction that represent the flow of information from the receiver of the corresponding caller message.



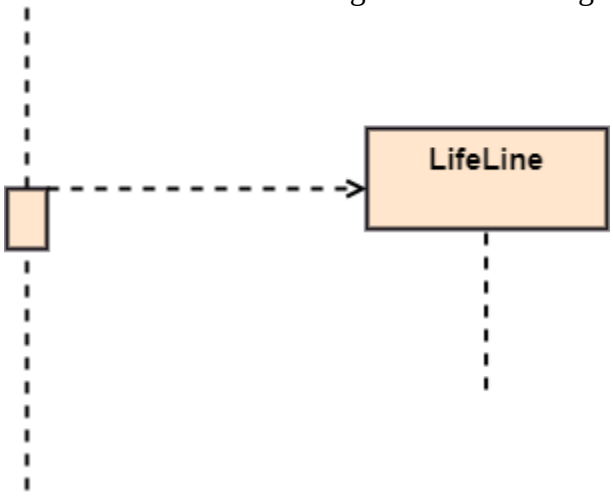
- **Self Message:** It describes a communication, particularly between the lifelines of an interaction that represents a message of the same lifeline, has been invoked.



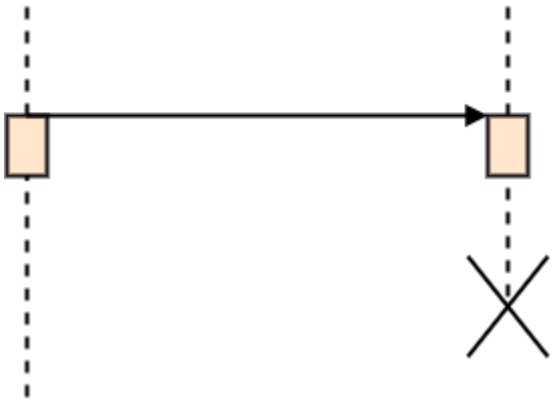
- **Recursive Message:** A self message sent for recursive purpose is called a recursive message. In other words, it can be said that the recursive message is a special case of the self message as it represents the recursive calls.



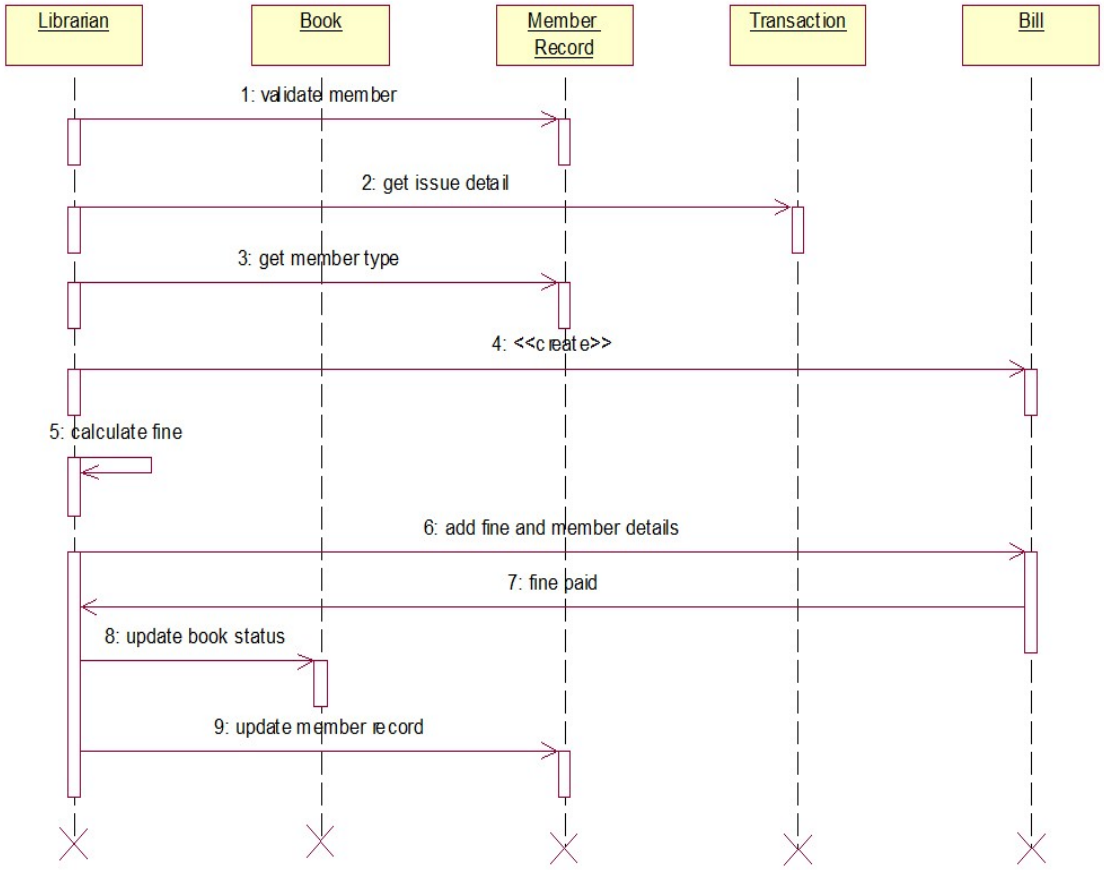
- **Create Message:** It describes a communication, particularly between the lifelines of an interaction describing that the target (lifeline) has been instantiated.



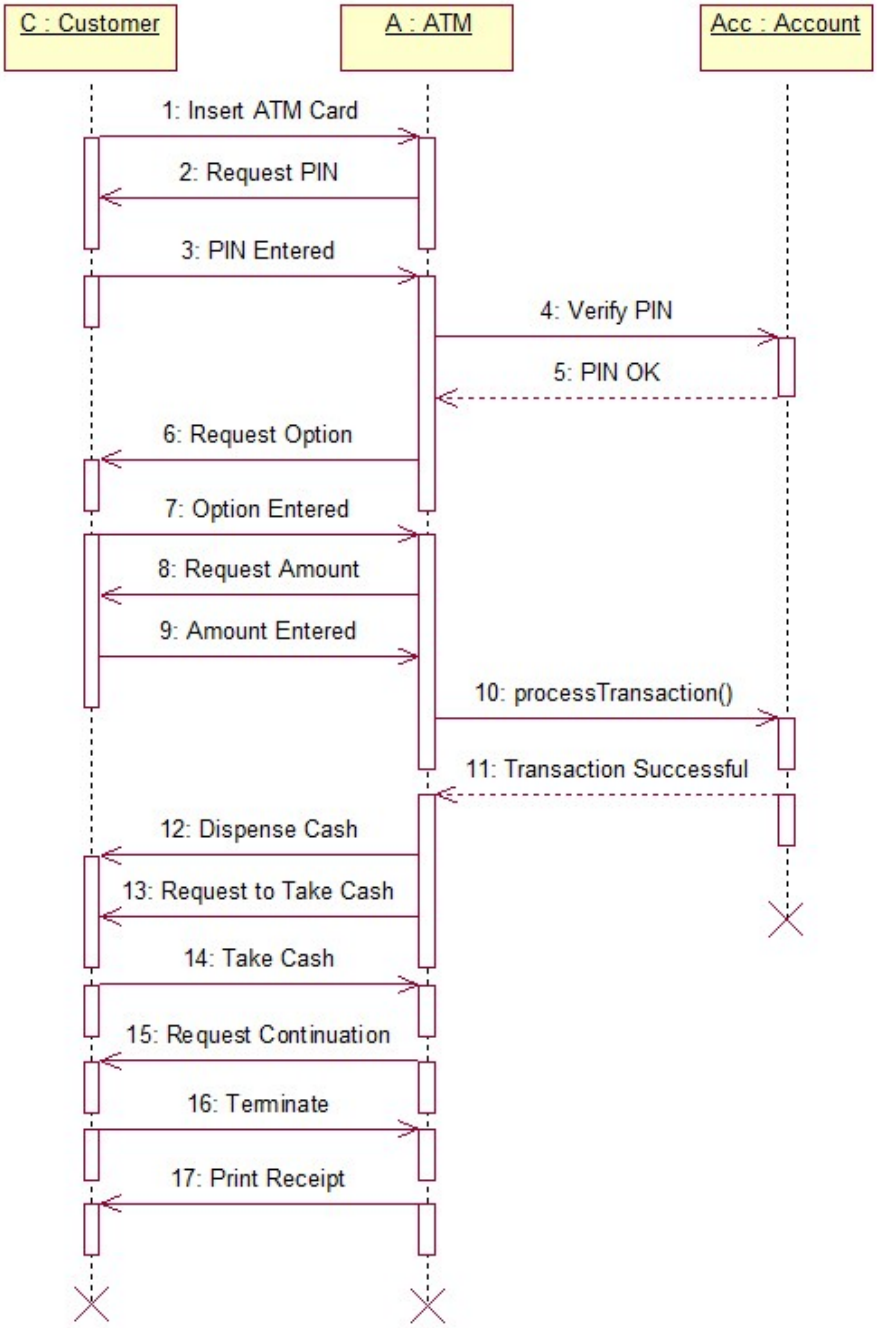
- **Destroy Message:** It describes a communication, particularly between the lifelines of an interaction that depicts a request to destroy the lifecycle of the target.



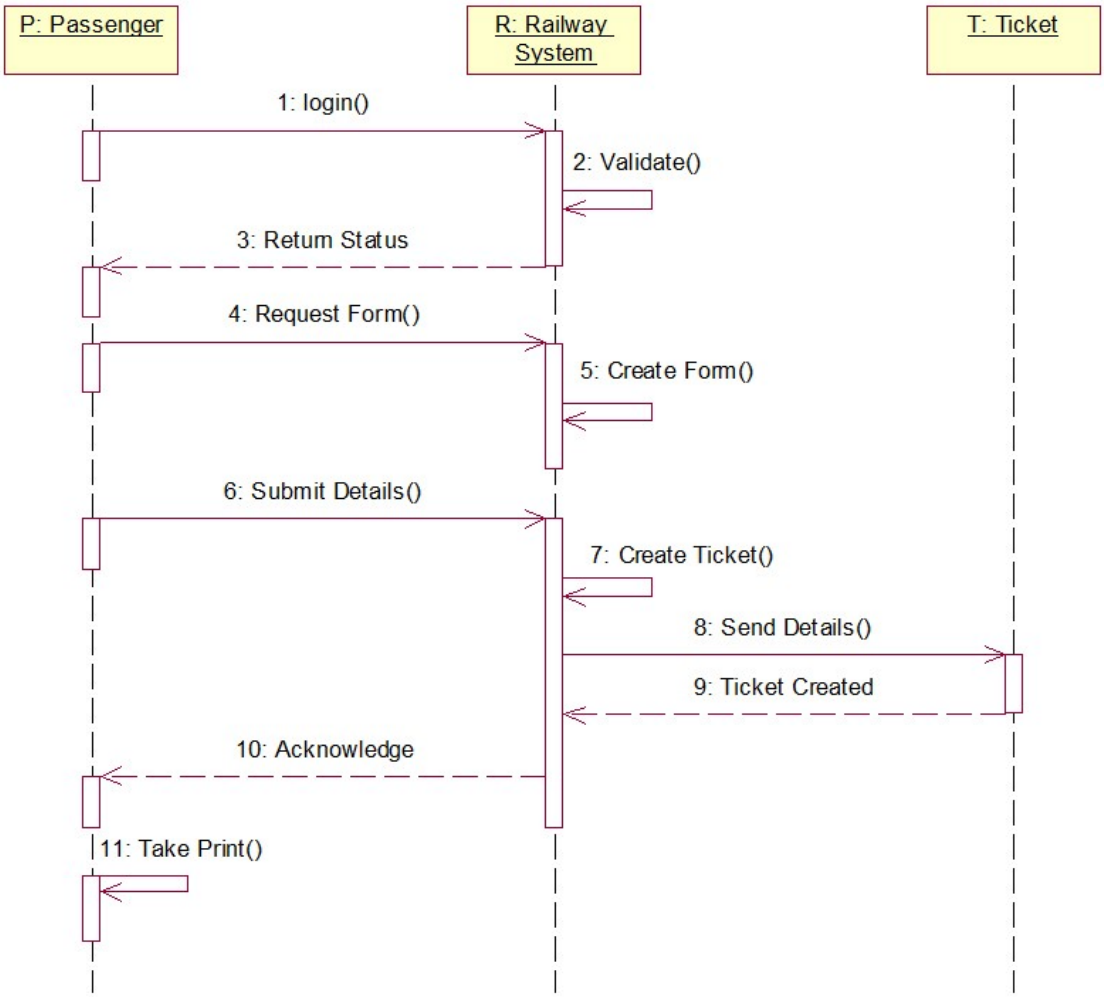
Library Management Systems



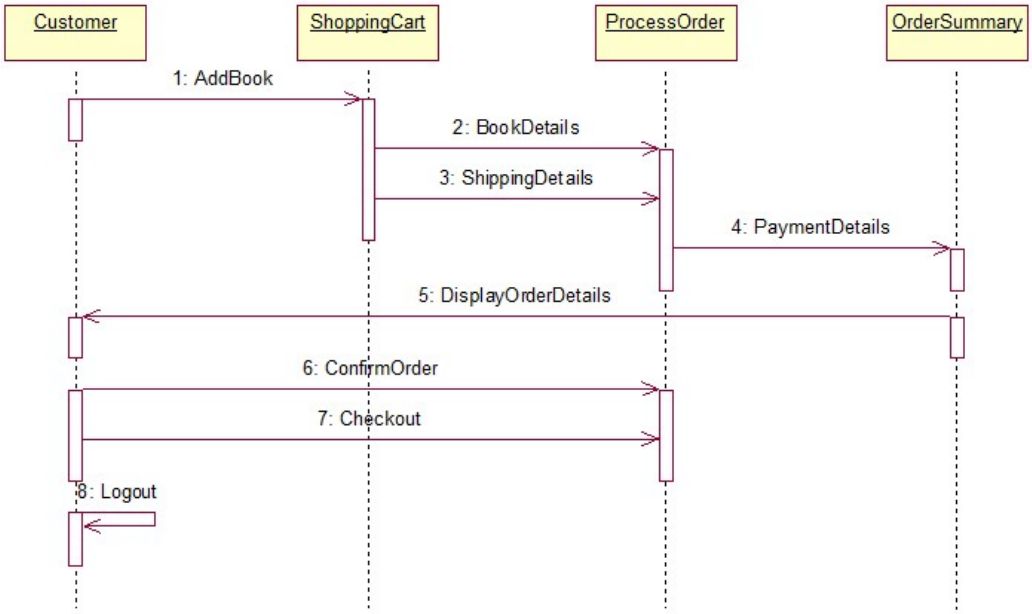
ATM



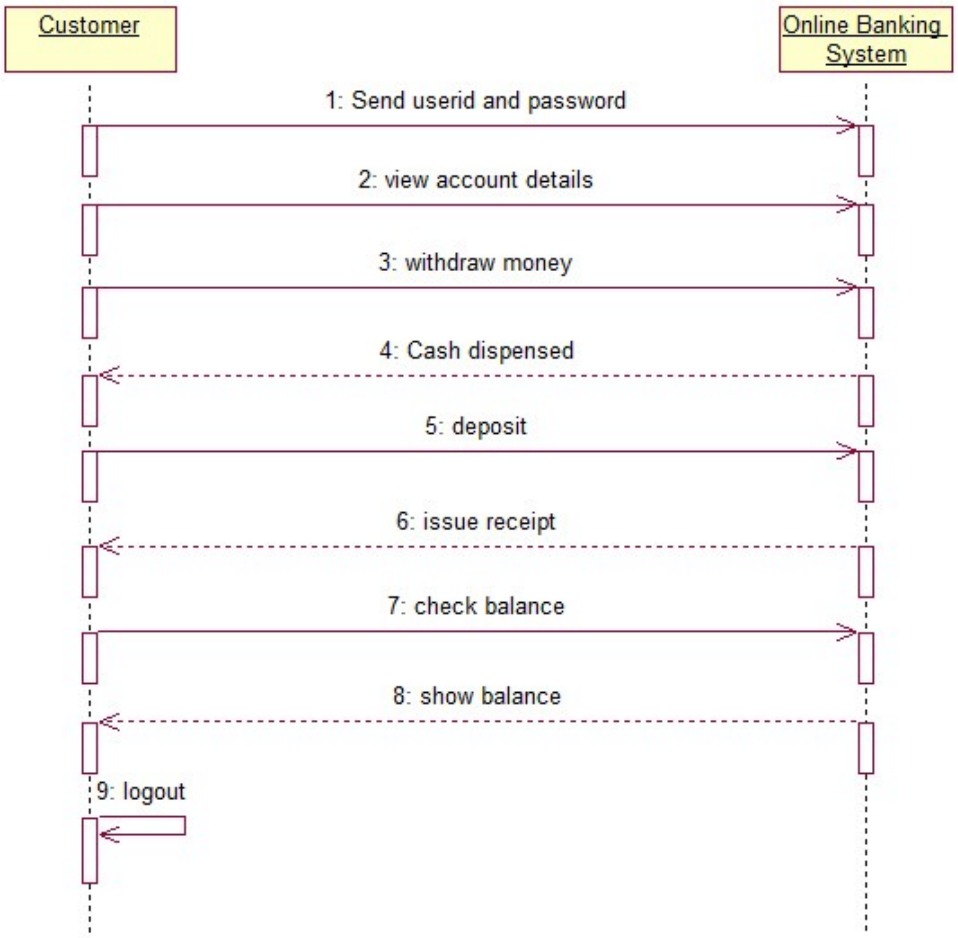
Railway Reservation



Online Book Shopping

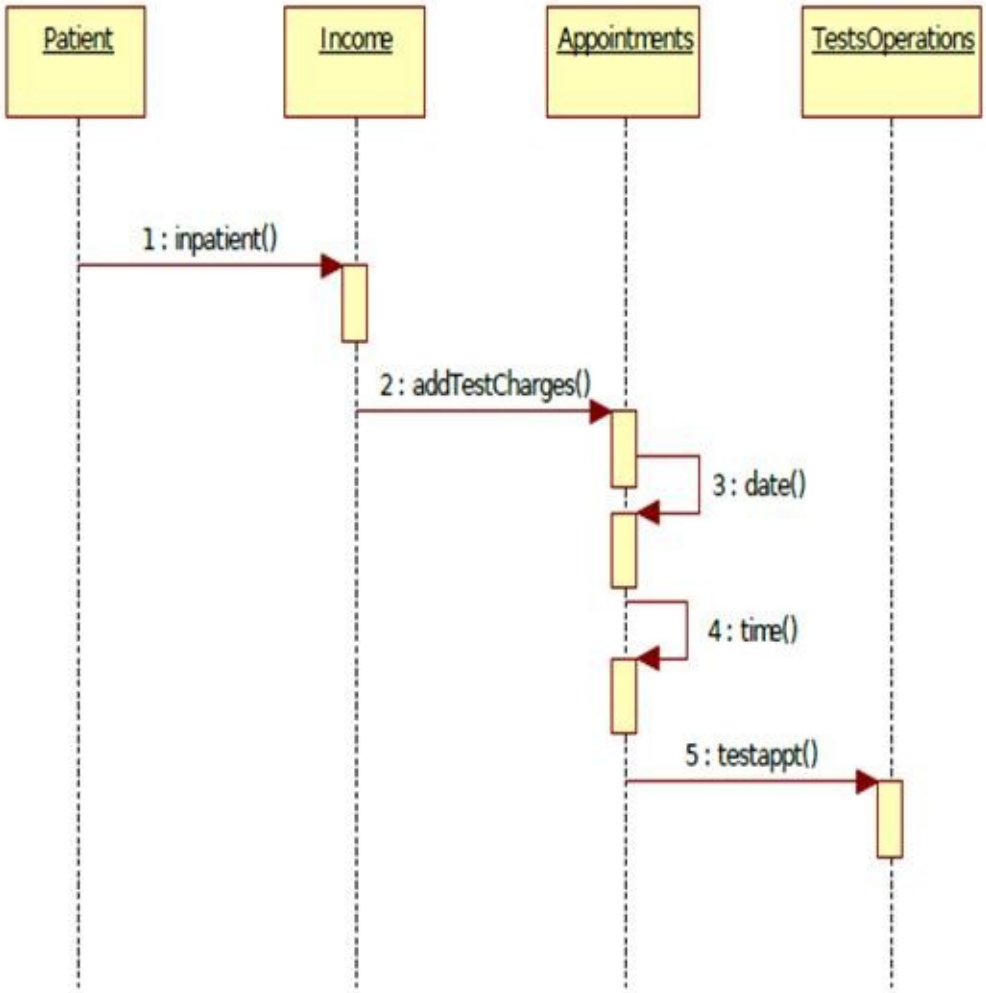


Online Banking

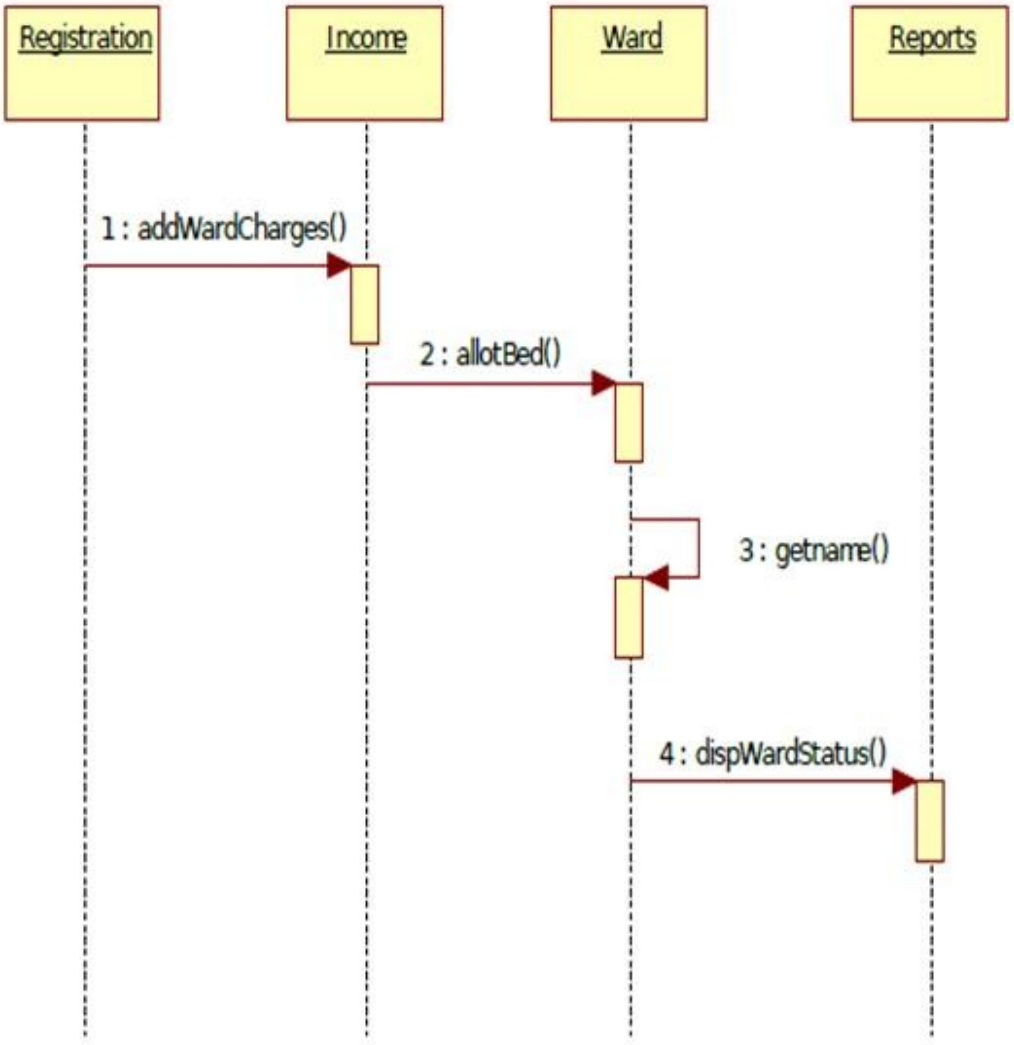


Hospital Management System

TESTS APPOINTMENTS:



BED ALLOTMENT:



Collaboration Diagram

What is the Collaboration Diagram?

Collaboration Diagram depicts the relationships and interactions among software objects. They are used to understand the object architecture within a system rather than the flow of a message as in a sequence diagram. They are also known as "Communication Diagrams."
As per Object-Oriented Programming (OOPs), an object entity has various attributes associated with it. Usually, there are multiple objects present inside an object-oriented system where each object can be associated with any other object inside the system. Collaboration Diagrams are used to explore the architecture of objects inside the system. The message flow between the objects can be represented using a collaboration diagram.

Benefits of Collaboration Diagram

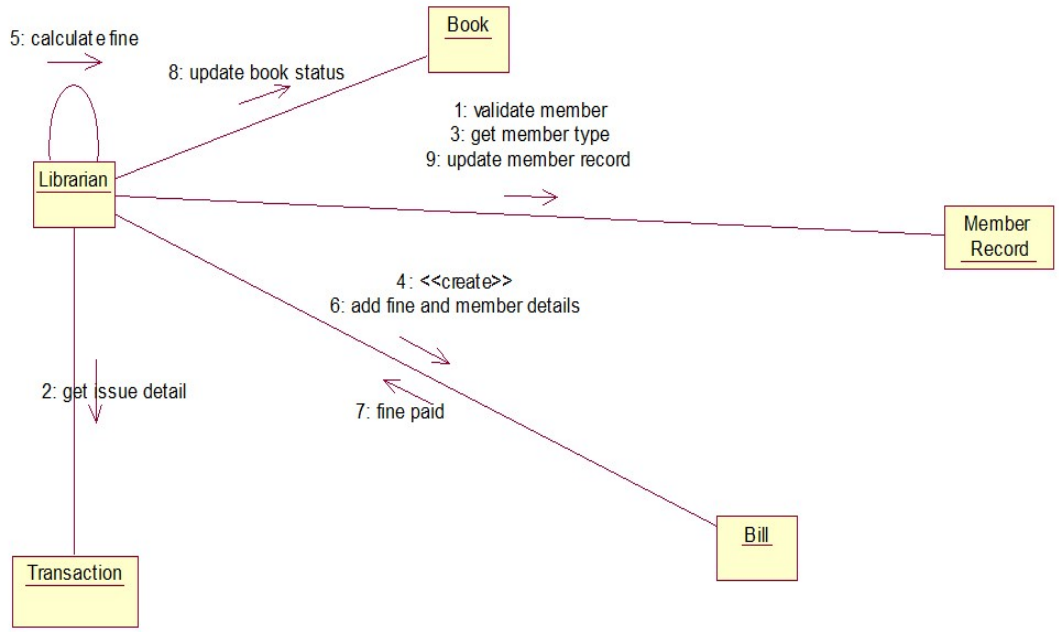
- It is also called as a communication diagram.
- It emphasizes the structural aspects of an interaction diagram – how lifeline connects.
- Its syntax is similar to that of sequence diagram except that lifeline don't have tails.
- Messages passed over sequencing is indicated by numbering each message hierarchically.
- Compared to the sequence diagram communication diagram is semantically weak.
- Object diagrams are special case of communication diagram.
- It allows you to focus on the elements rather than focusing on the message flow as described in the sequence diagram.
- Sequence diagrams can be easily converted into a collaboration diagram as collaboration diagrams are not very expressive.
- While modelling collaboration diagrams w.r.t sequence diagrams, some information may be lost.

The above collaboration diagram notation contains lifelines along with connectors, self-loops, forward, and reverse messages used in a collaboration diagram.

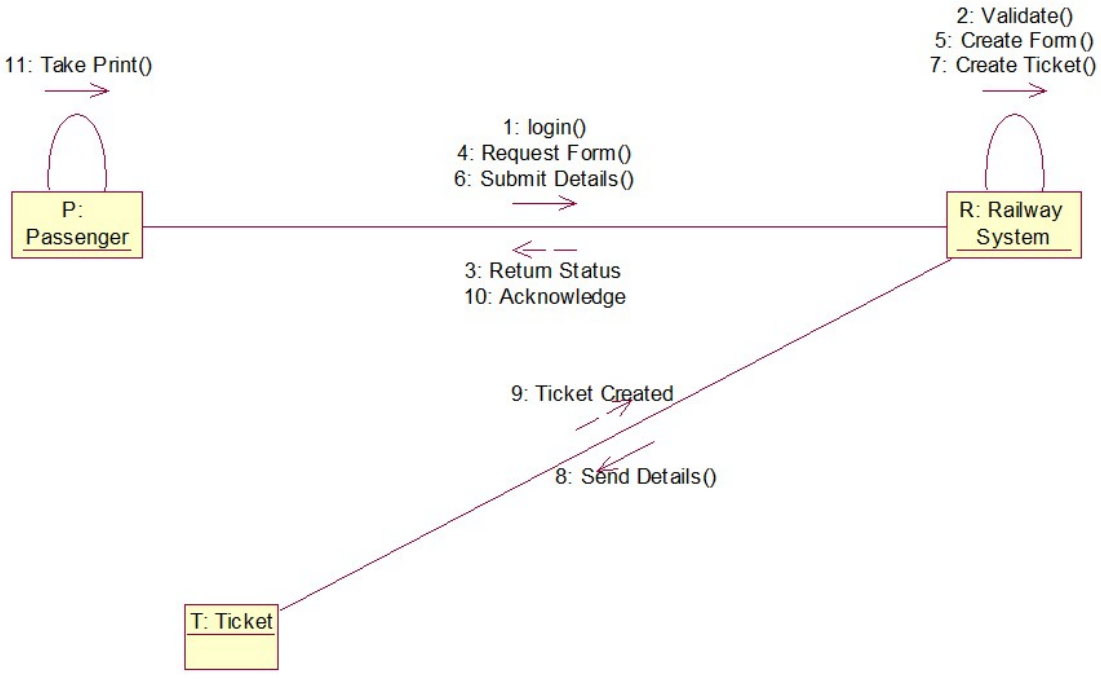
Drawbacks of a Collaboration Diagram

- Collaboration diagrams can become complex when too many objects are present within the system.
- It is hard to explore each object inside the system.
- Collaboration diagrams are time consuming.
- The object is destroyed after the termination of a program.
- The state of an object changes momentarily, which makes it difficult to keep track of every single change that occurs within an object of a system.

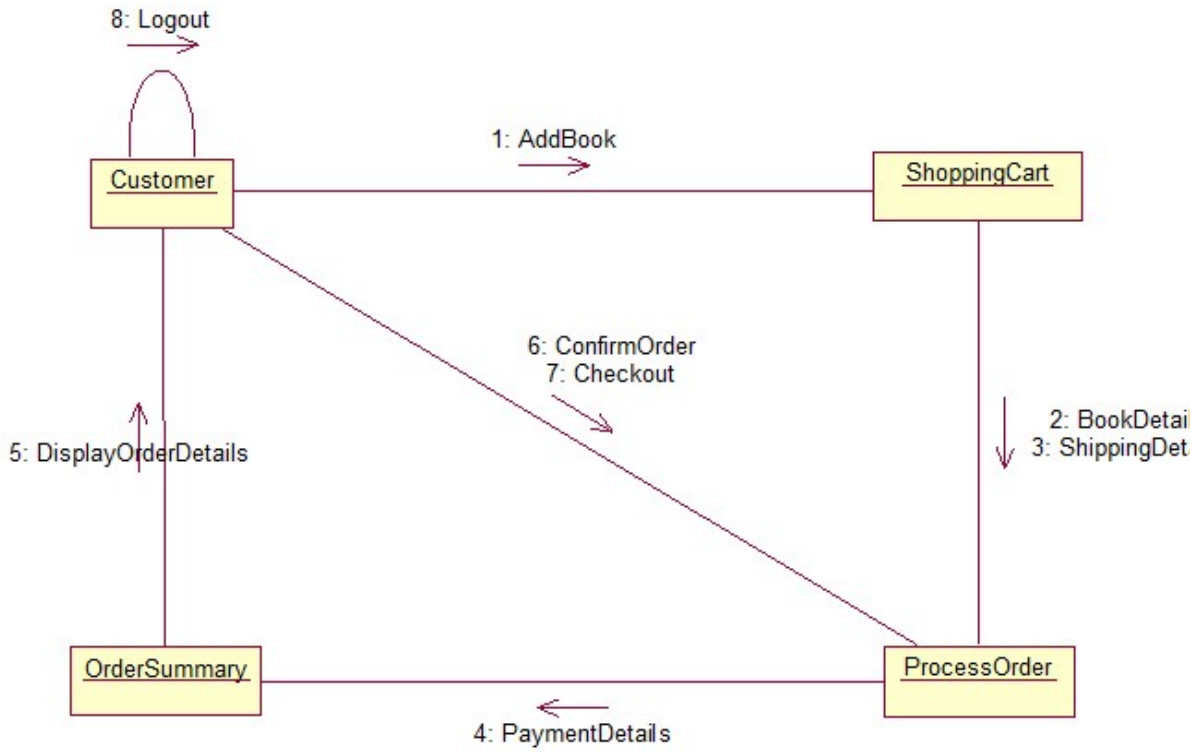
Library Management Systems



Railway Reservation

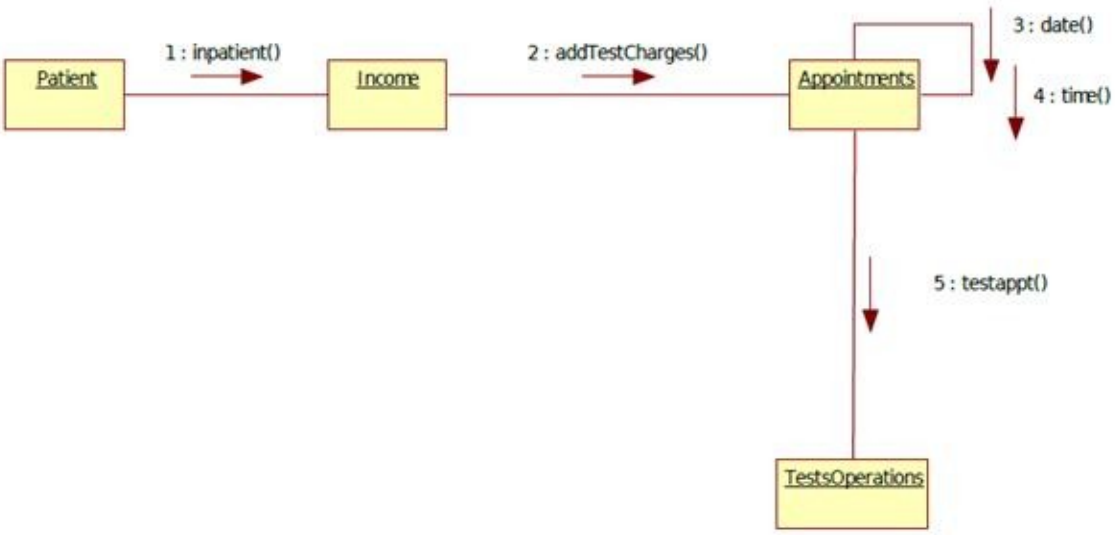


Online Book Shopping



Hospital Management System

TESTS APPOINTMENTS:



Class diagrams

Class Diagram
A Class Diagram is a type of structural diagram in the Unified Modeling Language (UML) that represents the static structure of a system. It shows the system's classes, their attributes, methods (operations), and the relationships among the classes.

Purpose

To provide a blueprint of the system from an object-oriented perspective.

To visualize, specify, construct, and document the components of a software system.

To serve as a reference model for developers and stakeholders.

To help in both forward engineering (design to code) and reverse engineering (code to design).

Key Components

Classes – Represent the building blocks of an object-oriented system. Each class defines:

Name – The identity of the class.

Attributes – The data members or properties of the class.

Operations – The functions or methods that the class can perform.

Visibility – Specifies whether members are public, private, protected, or package-level.

Relationships – Indicate how classes are connected or interact. Common relationships include:

Association – A link between classes.

Multiplicity – Specifies the number of objects involved in the relationship.

Aggregation – A "whole–part" relationship where the part can exist independently.

Composition – A stronger whole–part relationship where the part cannot exist without the whole.

Generalization (Inheritance) – Indicates that one class is a specialized form of another.

Dependency – A weaker relationship where one class relies on another.

Characteristics

Static in nature – Shows what exists in the system, not how it behaves over time.

Abstract view – Focuses on the logical design rather than implementation details.

Layered detail – Can be high-level for conceptual modeling or detailed for implementation modeling.

Importance in Software Engineering

Facilitates clear communication between designers, developers, and stakeholders.

Helps in requirement analysis by modeling real-world entities.

Assists in database design by mapping classes to tables.

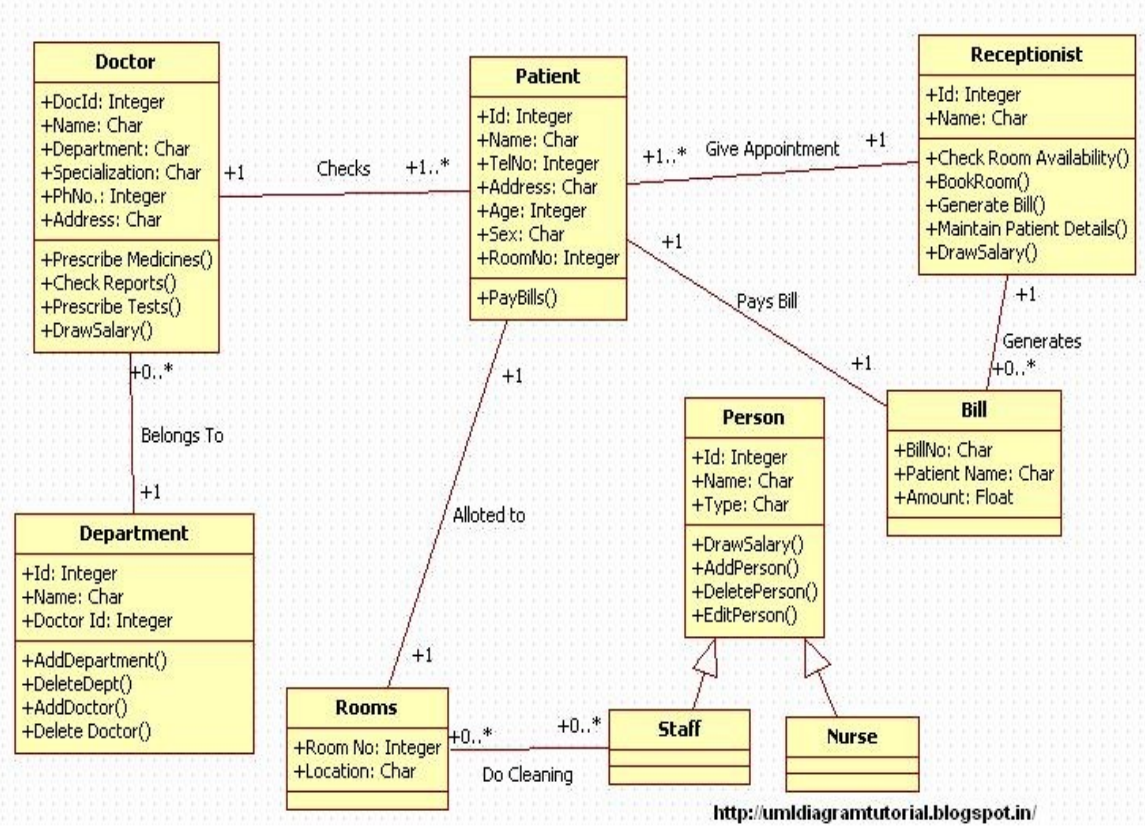
Supports maintainability by offering a quick reference to the system’s structure.

Limitations

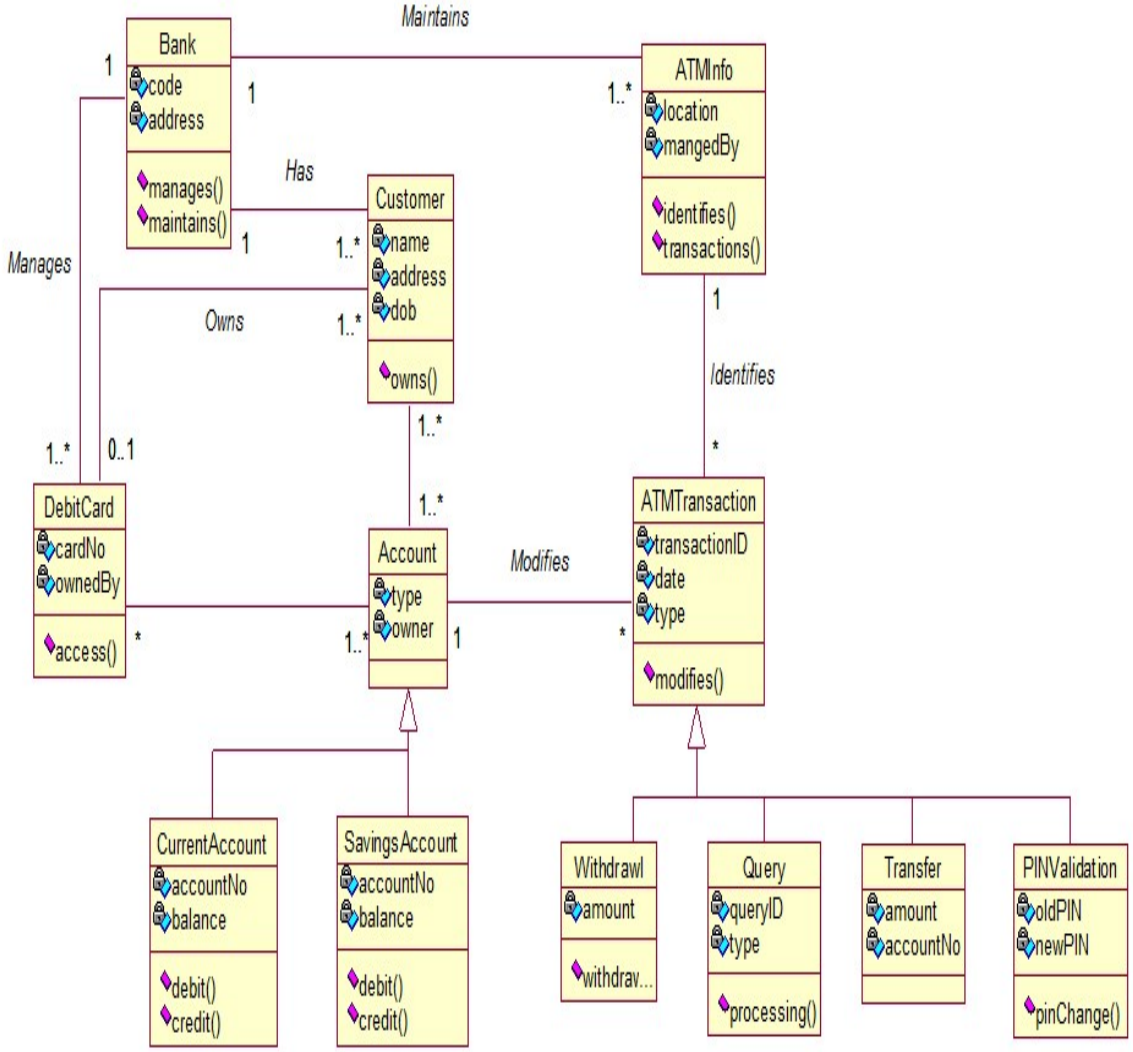
Cannot represent the dynamic behavior of the system (which is shown using sequence diagrams, activity diagrams, etc.).

Large systems can make class diagrams complex and harder to read.

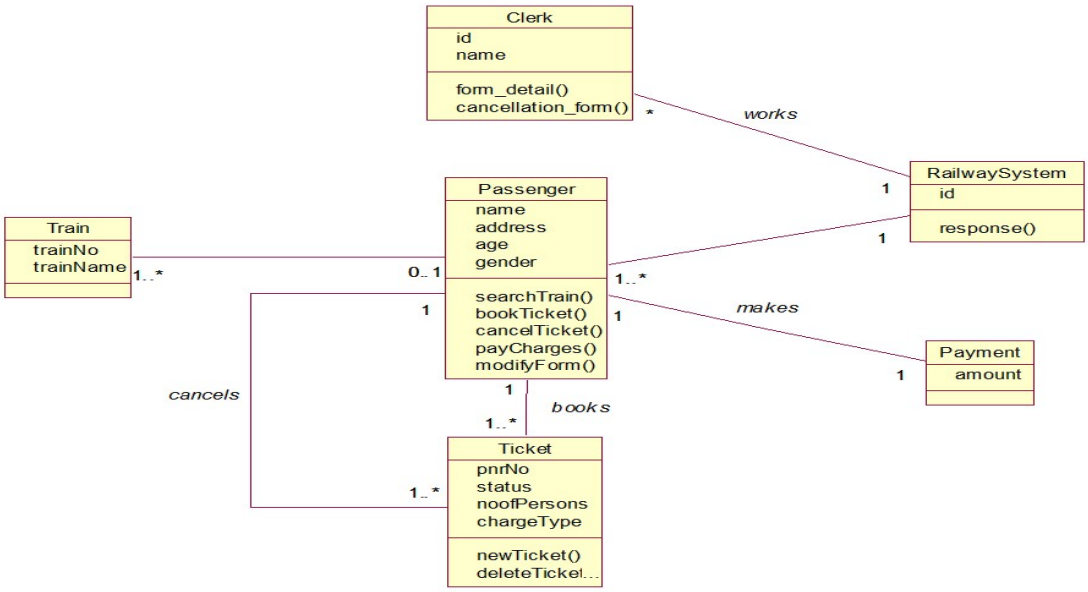
Hospital Management System



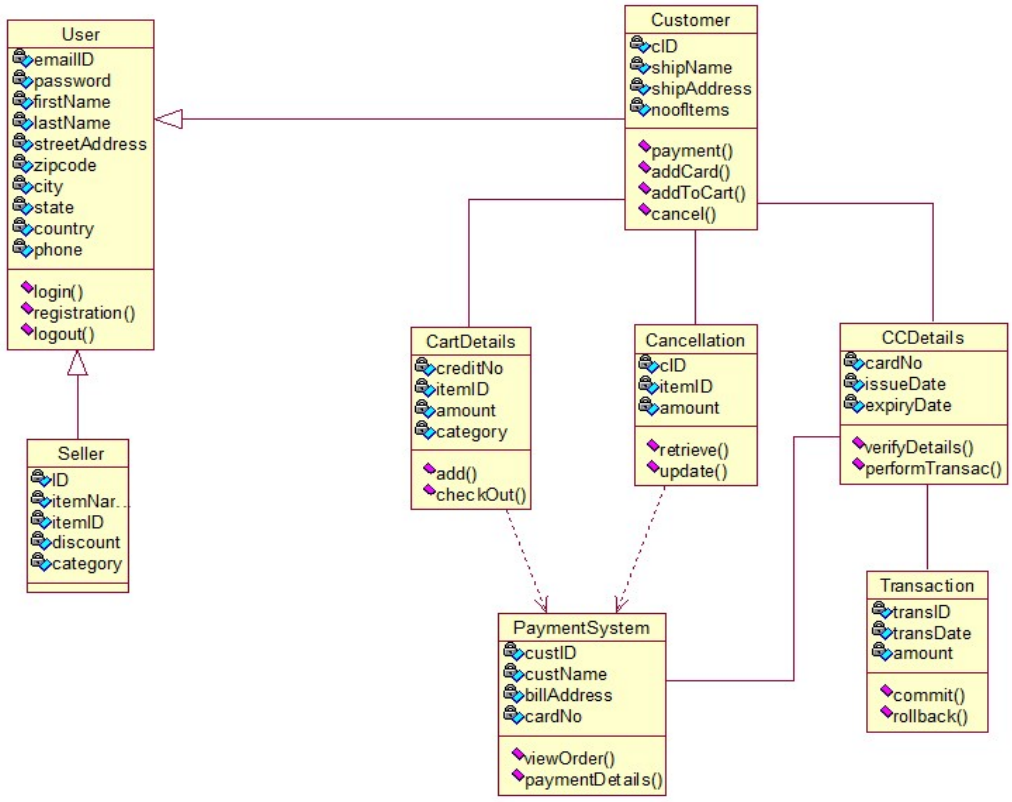
ATM


Maintains
Bank
code
address
manages()
maintains()
ATMInfo
location
mangedBy
identifies()
transactions()
Has
Customer
name
address
dob
owns()
Manages
Owns
Identifies
DebitCard
cardNo
ownedBy
access()
Account
type
owner
Modifies
ATMTransaction
transactionID
date
type
modifies()
CurrentAccount
accountNo
balance
debit()
credit()
SavingsAccount
accountNo
balance
debit()
credit()
Withdrawl
amount
withdrav...
Query
queryID
type
processing()
Transfer
amount
accountNo
PINValidation
oldPIN
newPIN
pinChange()

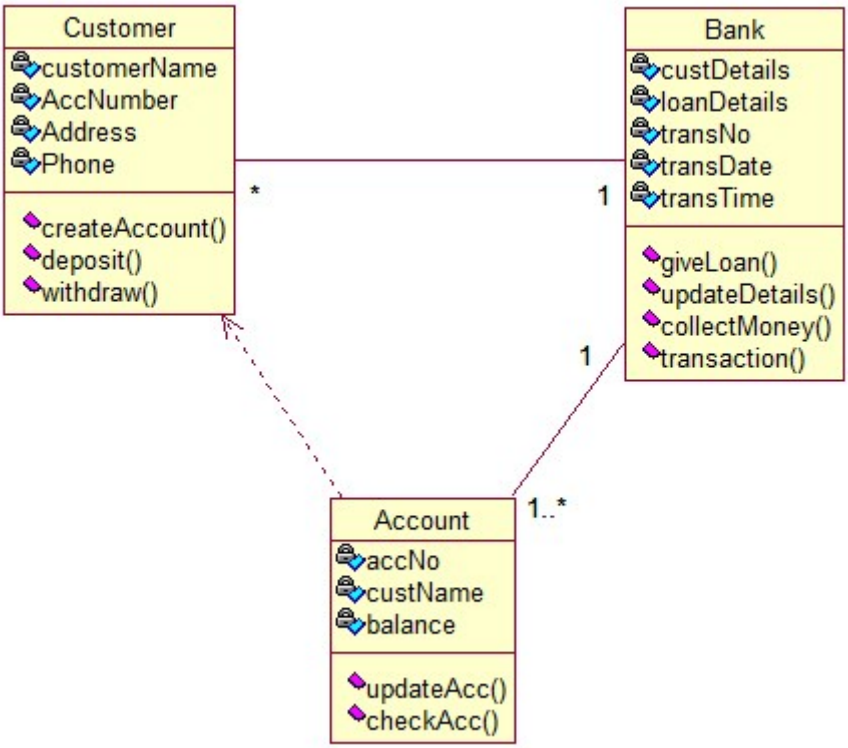
Railway Reservation



Online Book Shopping



Online Banking



State Chart Diagram

A state chart diagram in Unified Modeling Language (UML) is a behavioral diagram that depicts the various states that an object or system undergoes and the transitions between these states. It's particularly useful in modeling the behavior of an entity in response to external or internal events.

1. Definition

A Statechart Diagram (also called a State Machine Diagram) in UML describes the dynamic behavior of a system by showing:

The different states of an object

Transitions between these states

Events and conditions that trigger these transitions

It is particularly useful for modeling reactive systems — systems whose behavior depends on events or conditions.

2. Purpose

To model the lifecycle of a single object.

To represent how an object changes its state in response to events.

To capture the sequence of states, conditions, and actions.

To help in designing event-driven systems such as embedded systems, workflow processes, or UI navigation

3. Elements of a Statechart Diagram

a) State

Represents a condition or situation during the life of an object.

Drawn as a rounded rectangle.

Types of states:

Initial State: Shown as a filled black circle; the starting point.

Final State: Shown as a filled black circle inside a hollow circle; the end point.

Simple State: A single condition.

Composite State: Contains substates.

b) Transition

A directed arrow showing movement from one state to another.

Triggered by an event (and optionally guarded by a condition)

Syntax:
event [guard condition] / action

Example:
buttonPressed [userLoggedIn] / displayMenu()

c) Event

Something that happens and causes a change in state (e.g., signal, message, condition becoming true).

d) Action

Activity that occurs as a result of a transitions

4. Notation

Initial state: ●

Final state: ◎

State: Rounded rectangle with state name inside.

Transition: Arrow from one state to another with event/action label.

5. Example Scenario

ATM Withdrawal Process

Initial State: Idle

States: Card Inserted → PIN Entered → Transaction Selected → Cash Dispensed

Final State: Session Ended

Transitions occur due to events like insertCard, enterPIN, selectWithdrawal, dispenseCash.

6. Advantages

Helps in visualizing event-driven behavior.

Ensures all possible states and transitions are considered.

Useful in testing, since it shows all valid and invalid transitions.

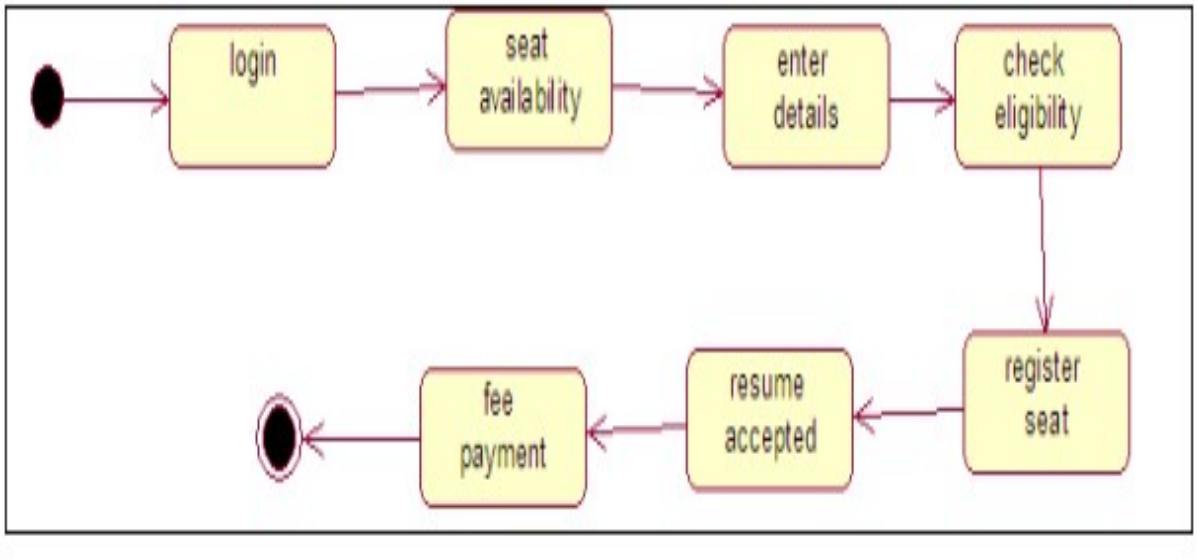
7. Limitations

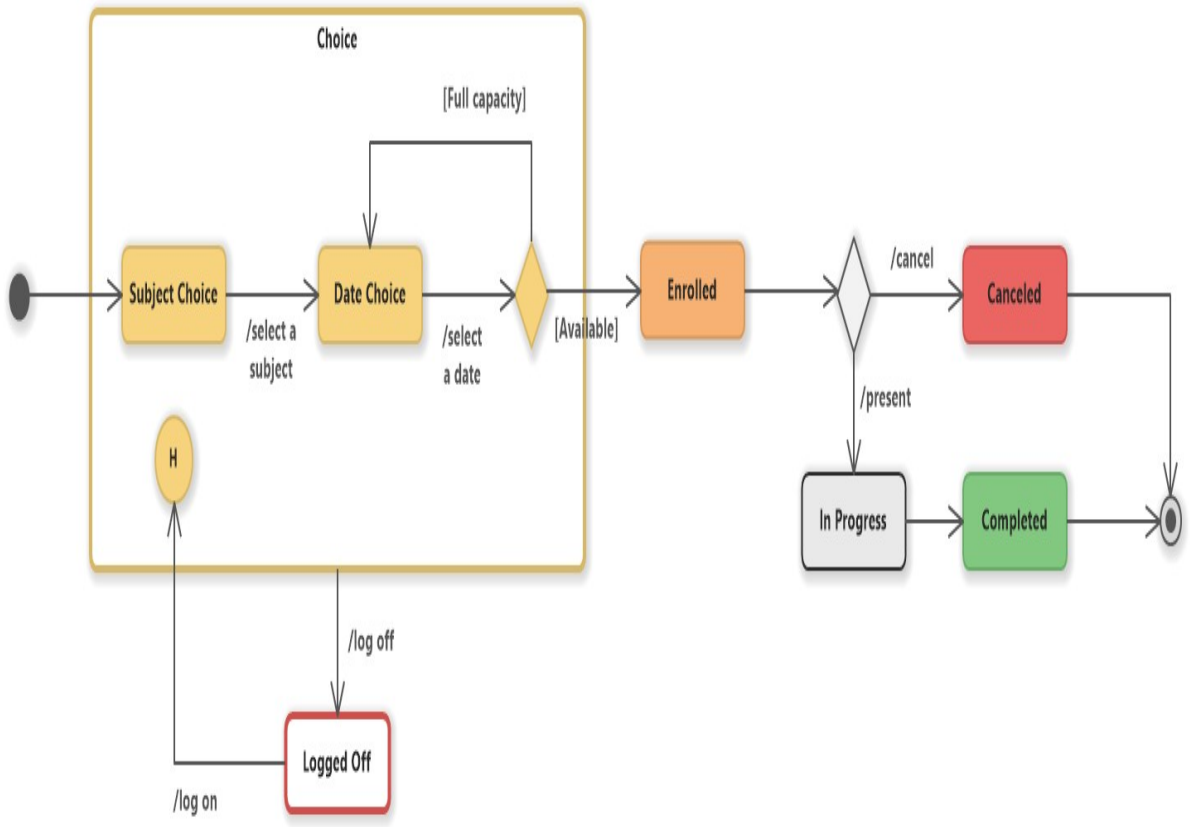
Can become complex for large systems.

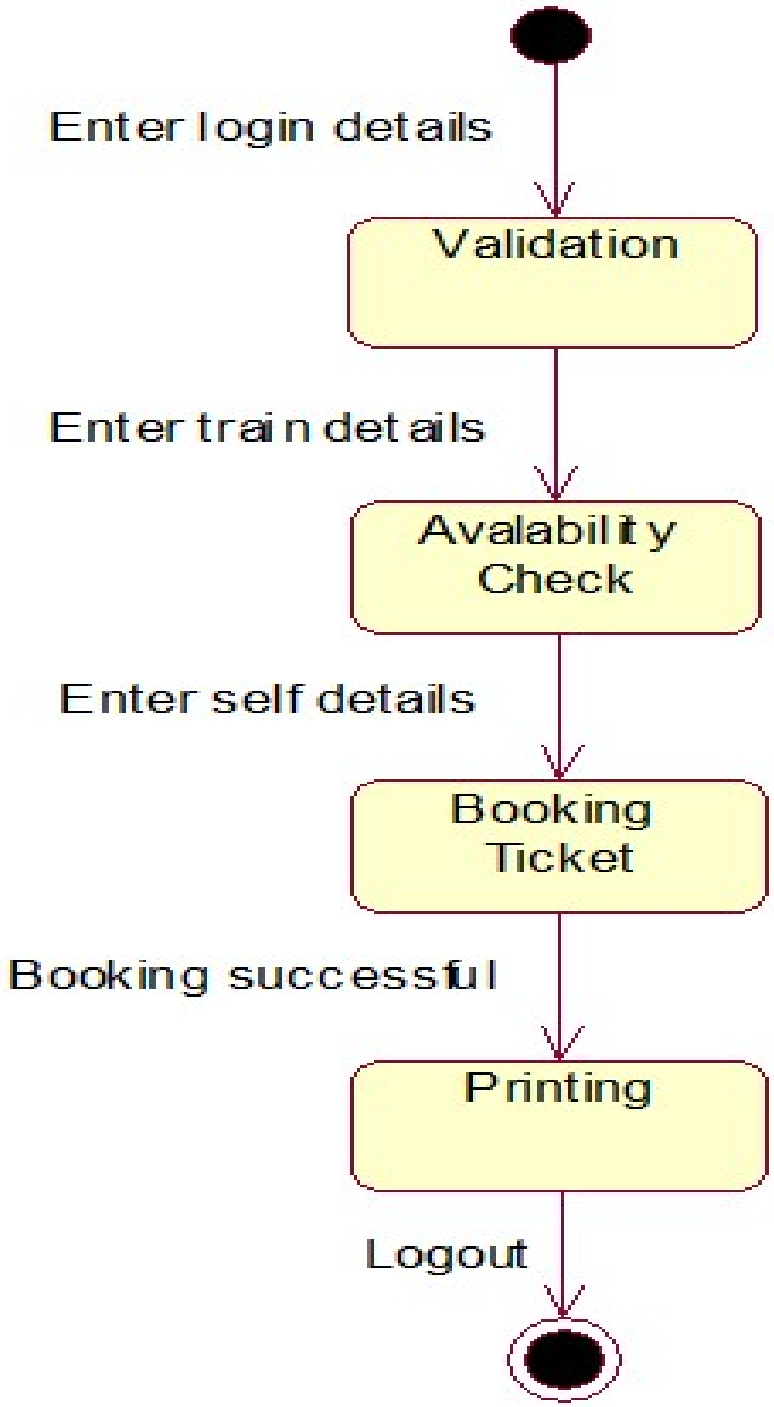
Not suitable for modeling static structure.

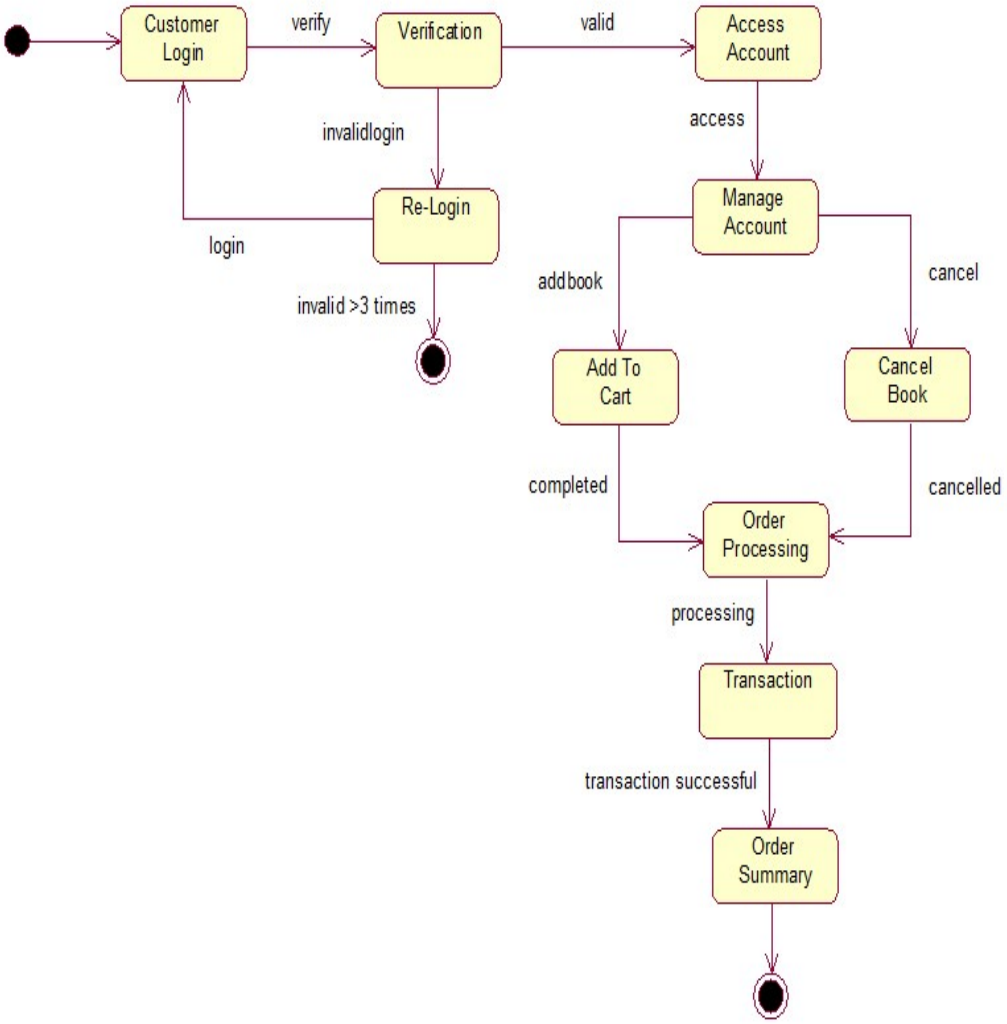
Here is an example of a simple state chart diagram:

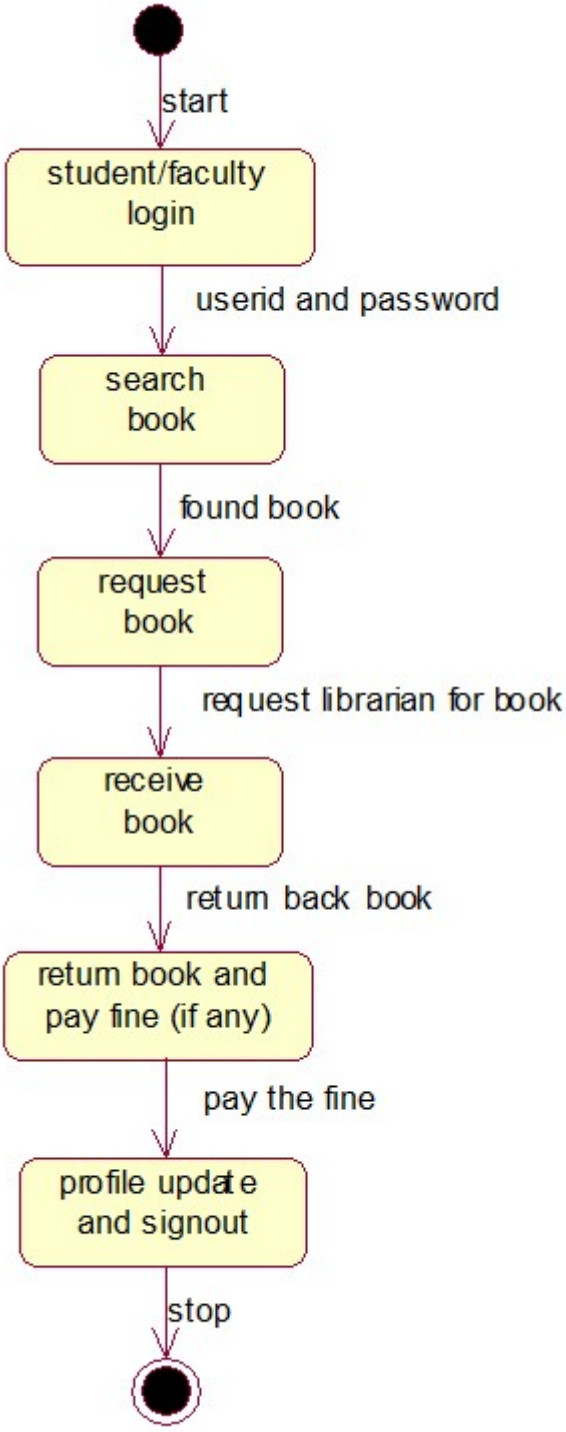
Course registration

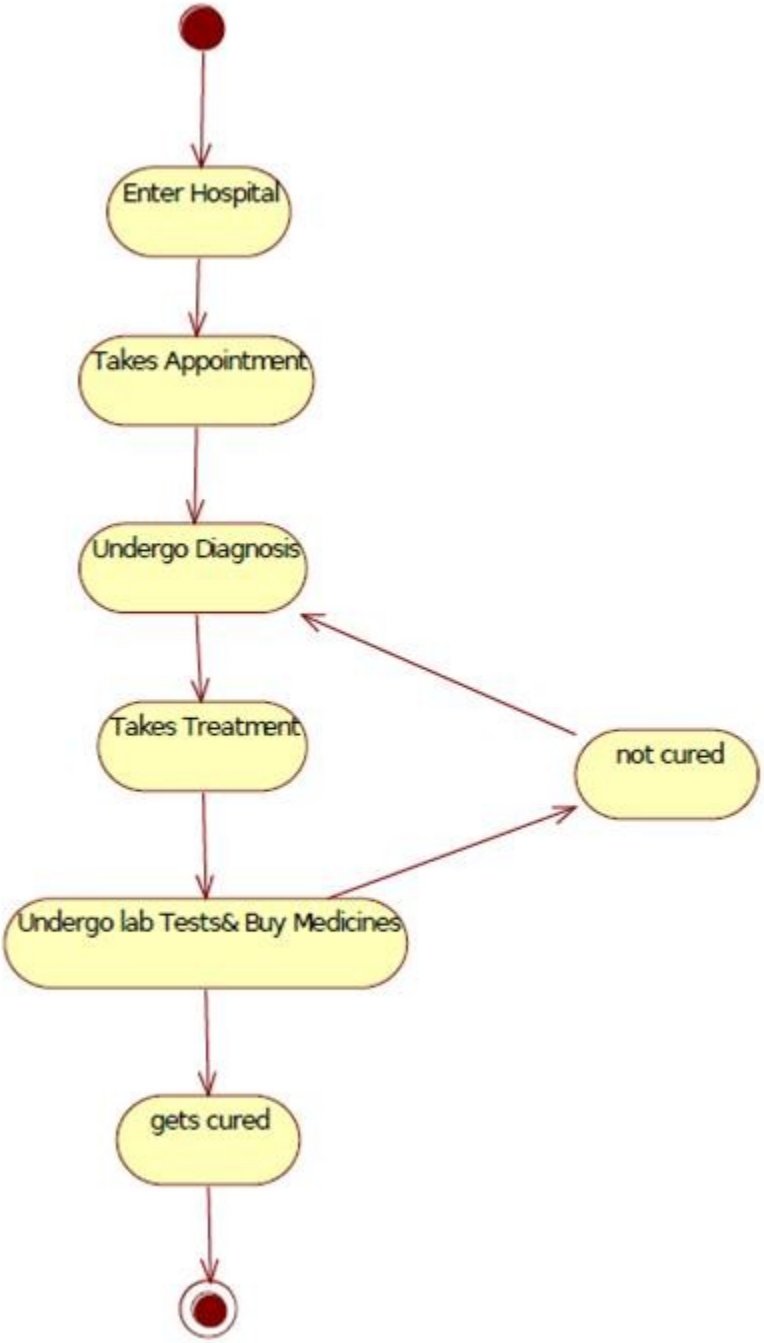



Choice
[Full capacity]
Subject Choice
/select a subject
Date Choice
/select a date
[Available]
Enrolled
/cancel
Canceled
/present
In Progress
Completed
H
/log off
Logged Off
/log on


Enter login details
Validation
Enter train details
Avalability Check
Enter self details
Booking Ticket
Booking successful
Printing
Logout

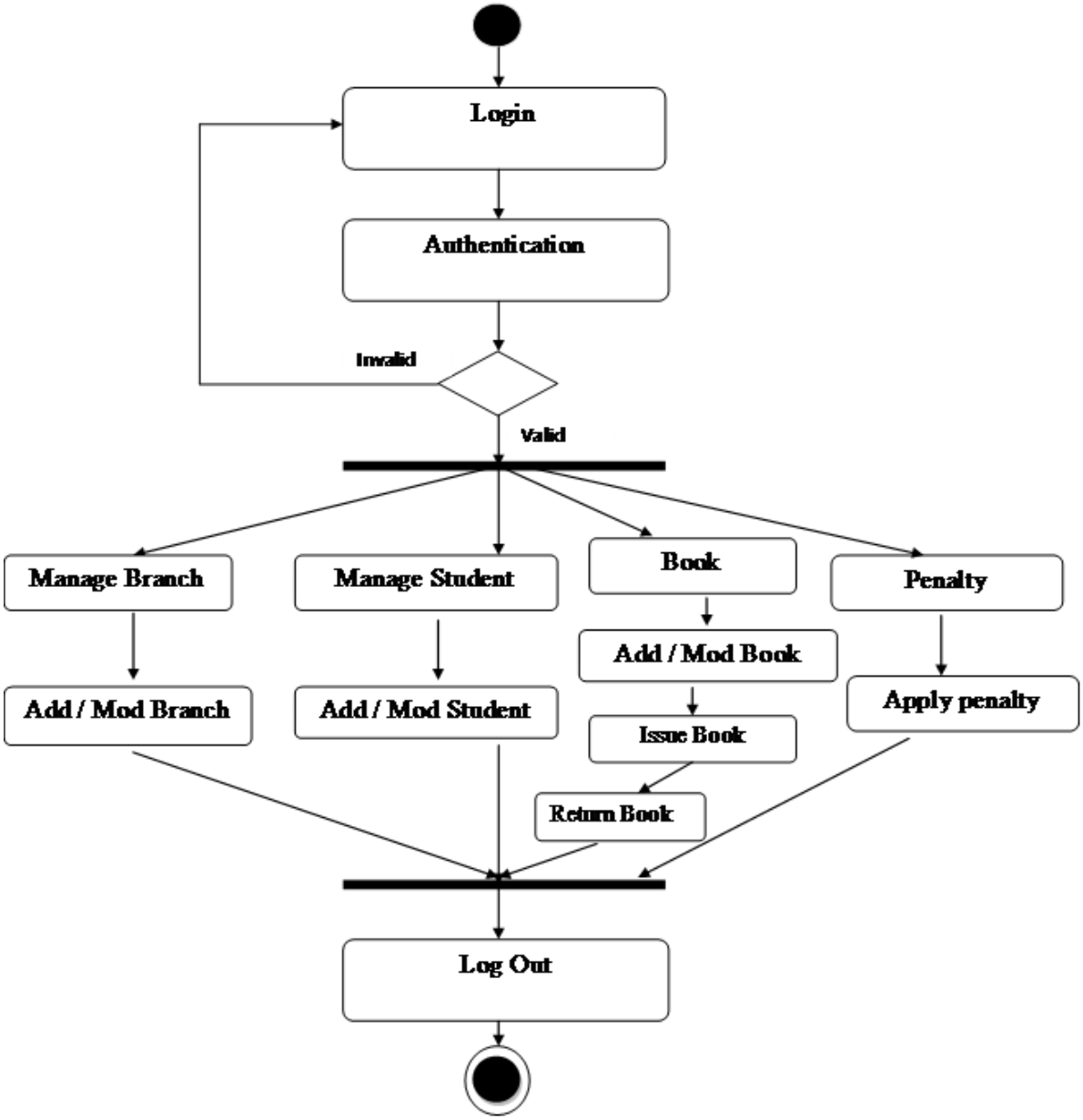

Customer Login
verify
Verification
valid
Access Account
invalidlogin
access
Re-Login
Manage Account
login
addbook
cancel
invalid >3 times
Add To Cart
Cancel Book
completed
cancelled
Order Processing
processing
Transaction
transaction successful
Order Summary


start
student/faculty login
userid and password
search book
found book
request book
request librarian for book
receive book
return back book
return book and pay fine (if any)
pay the fine
profile update and signout
stop

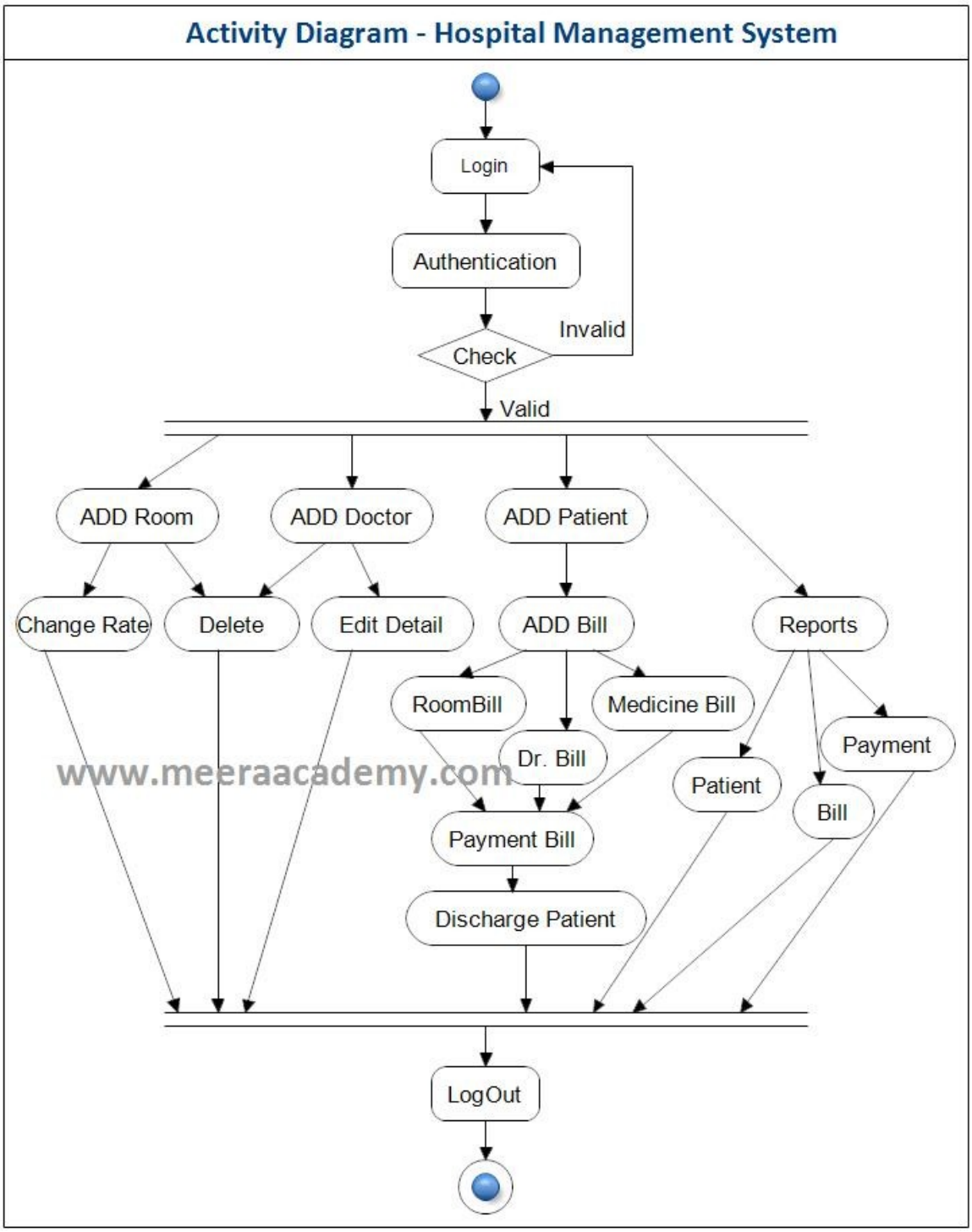

Enter Hospital
Takes Appointment
Undergo Diagnosis
Takes Treatment
not cured
Undergo lab Tests& Buy Medicines
gets cured

8. Activity Diagram

Activity Diagrams to illustrate the flow of control in a system and refer to the steps involved in the execution of a use case. We model sequential and concurrent activities using activity diagrams. So, we basically depict workflows visually using an activity diagram. An activity diagram focuses on condition of flow and the sequence in which it happens. We describe or depict what causes a particular event using an activity diagram. UML models basically three types of diagrams, namely, structure diagrams, interaction diagrams, and behavior diagrams. An activity diagram is a **behavioral diagram** i.e. it depicts the behavior of a system. An activity diagram portrays the control flow from a start point to a finish point showing the various decision paths that exist while the activity is being executed. We can depict both sequential processing and concurrent processing of activities using an activity diagram. They are used in business and process modelling where their primary use is to depict the dynamic aspects of a system. An activity diagram is very **similar to a flowchart**.

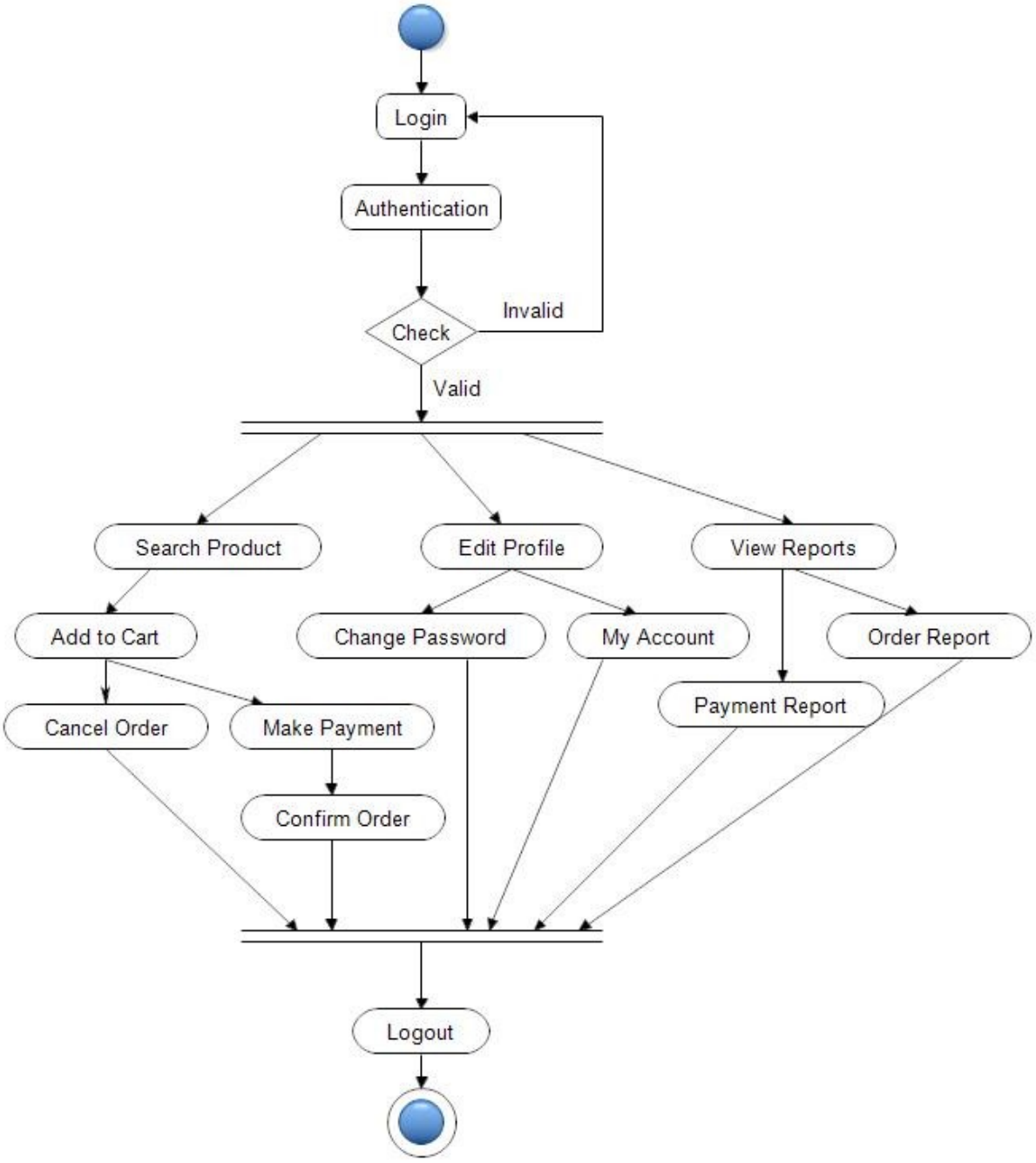

Login
Authentication
Invalid
Valid
Manage Branch
Manage Student
Book
Penalty
Add / Mod Book
Add / Mod Branch
Add / Mod Student
Issue Book
Apply penalty
Return Book
Log Out

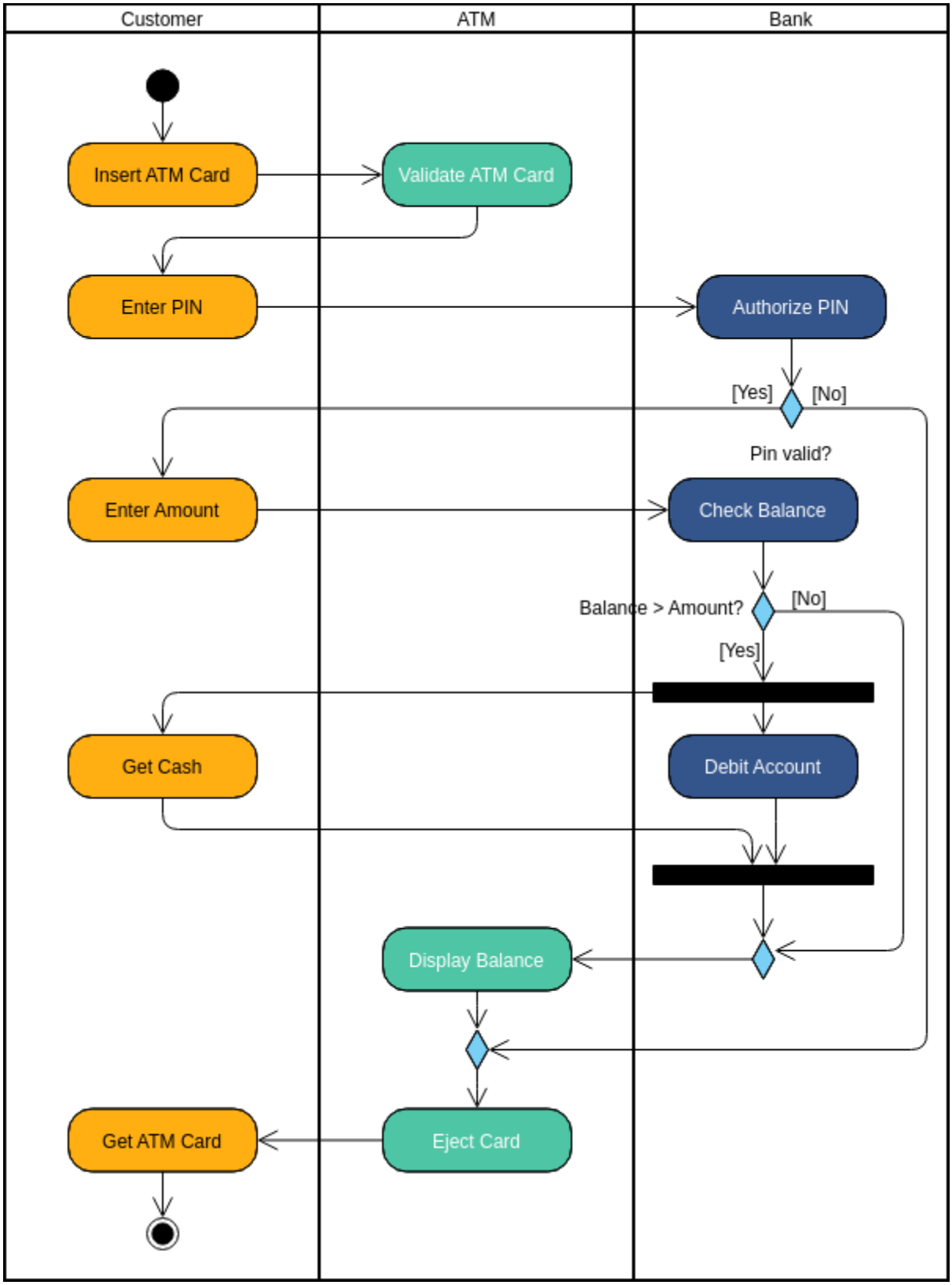
Activity Diagram - Hospital Management System



www.meeraacademy.com

Activity Diagram for User Side



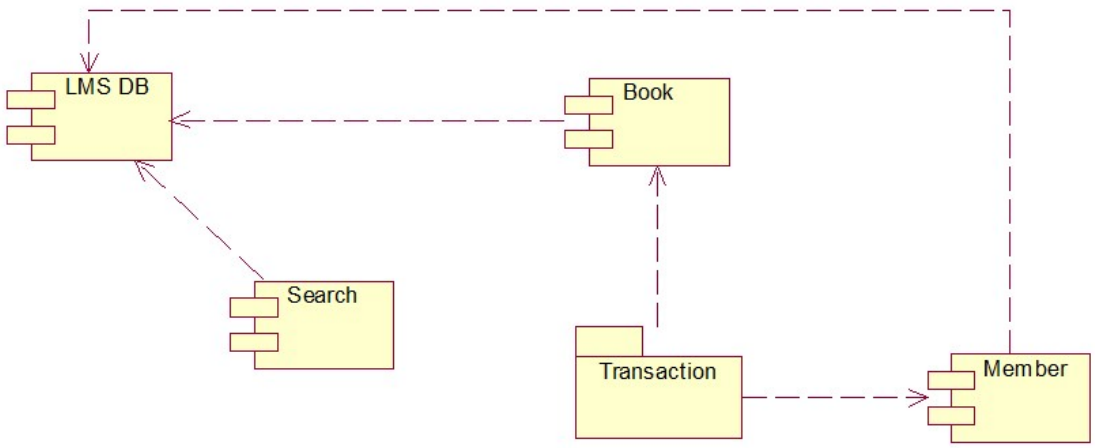

Customer
ATM
Bank
Insert ATM Card
Validate ATM Card
Enter PIN
Authorize PIN
[Yes]
[No]
Pin valid?
Enter Amount
Check Balance
Balance > Amount?
[No]
[Yes]
Get Cash
Debit Account
Display Balance
Get ATM Card
Eject Card

9.Component Diagram

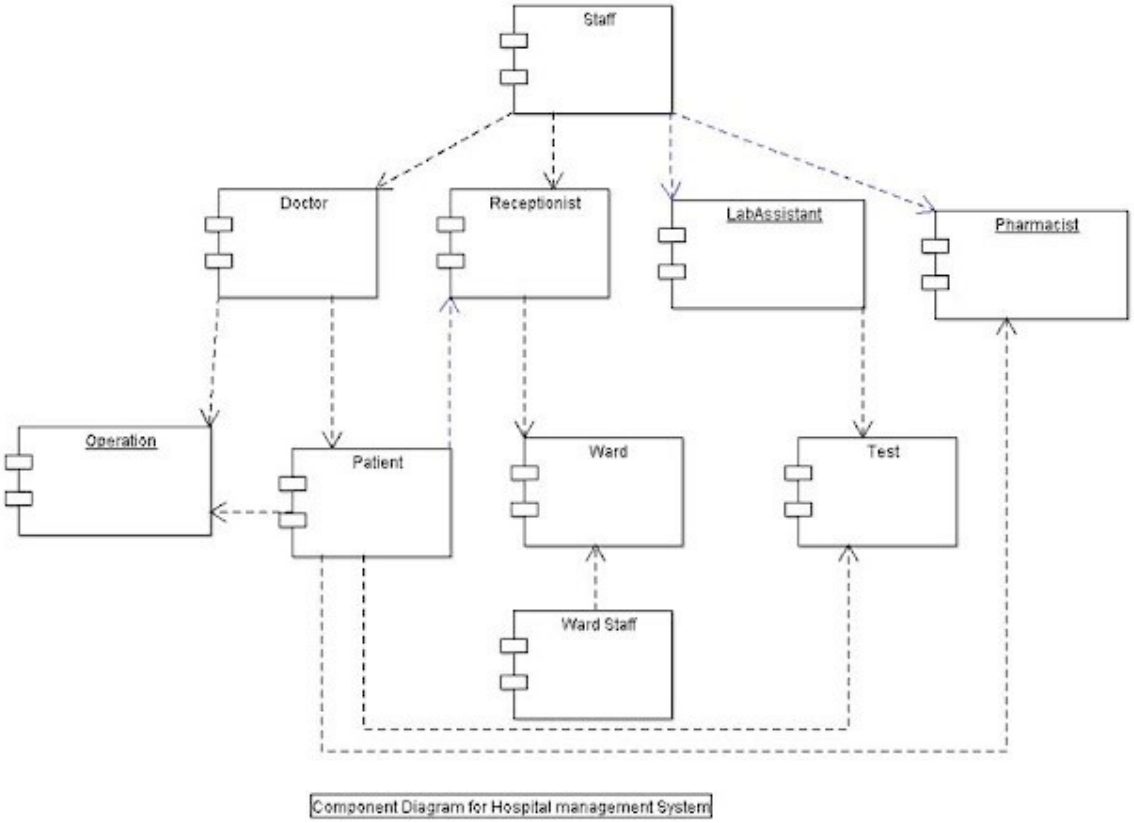
A component diagram defines the composition of components and artifacts in the system. A deployment diagram shows components and artifacts in relation to where they are used in the deployed system. In UML modeling, components are model elements that represent independent, interchangeable parts of a system. A component diagram is used to break down a large object-oriented system into the smaller components, so as to make them more manageable. It models the physical view of a system such as executables, files, libraries, etc. that resides within the node.

It visualizes the relationships as well as the organization between the components present in the system. It helps in forming an executable system. A component is a single unit of the system, which is replaceable and executable. The implementation details of a component are hidden, and it necessitates an interface to execute a function. It is like a black box whose behavior is explained by the provided and required interfaces.

Library Management System:

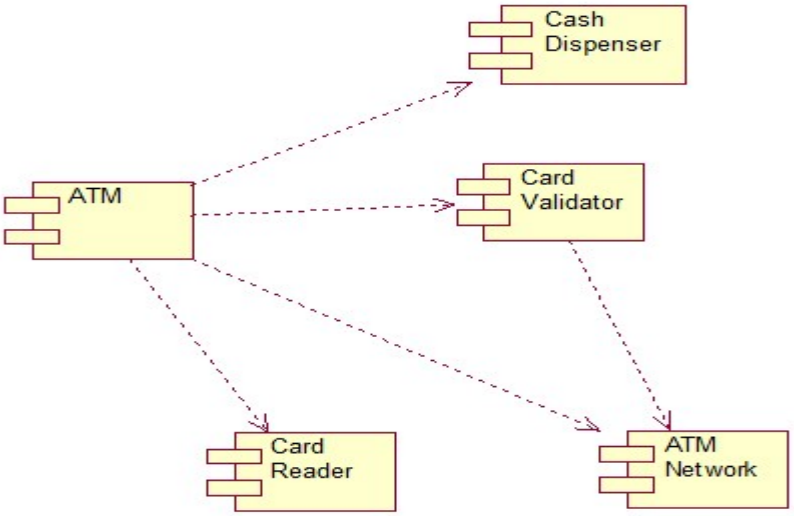


Hospital Management System

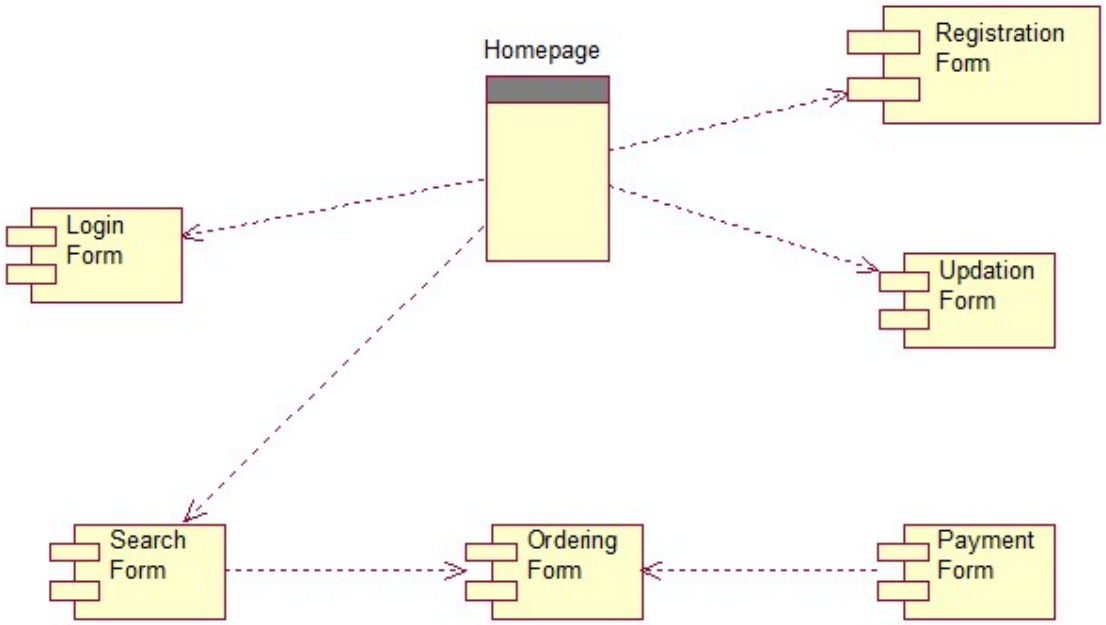


Component Diagram for Hospital management System

ATM



Online Shopping



Deployment Diagram

UML Deployment Diagram

The deployment diagram visualizes the physical hardware on which the software will be deployed. It portrays the static deployment view of a system. It involves the nodes and their relationships.

It ascertains how software is deployed on the hardware. It maps the software architecture created in design to the physical system architecture, where the software will be executed as a node. Since it involves many nodes, the relationship is shown by utilizing communication paths.

Purpose of Deployment Diagram

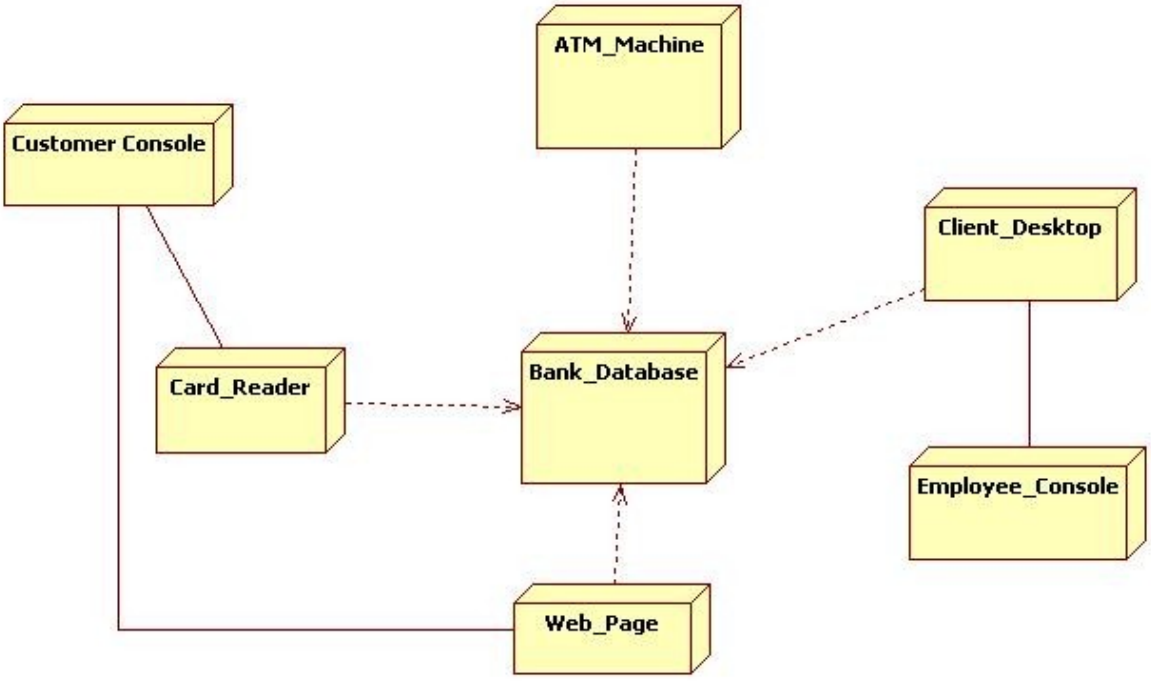
The main purpose of the deployment diagram is to represent how software is installed on the hardware component. It depicts in what manner a software interacts with hardware to perform its execution.

Both the deployment diagram and the component diagram are closely interrelated to each other as they focus on software and hardware components. The component diagram represents the components of a system, whereas the deployment diagram describes how they are actually deployed on the hardware.

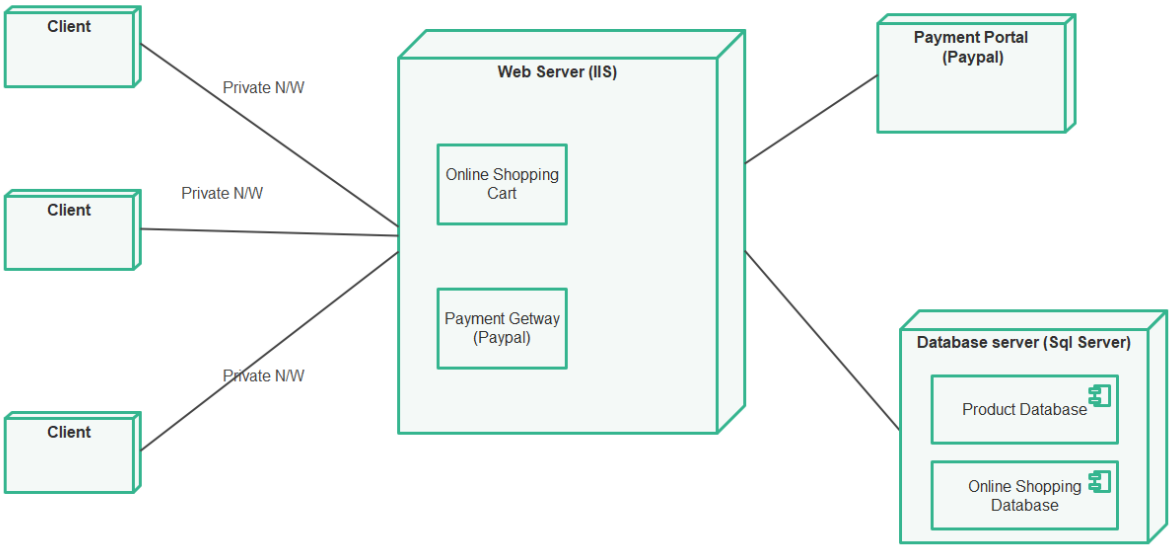
Following are the purposes of deployment diagram enlisted below:

1. To envision the hardware topology of the system.
2. To represent the hardware components on which the software components are installed.
3. To describe the processing of nodes at the runtime.

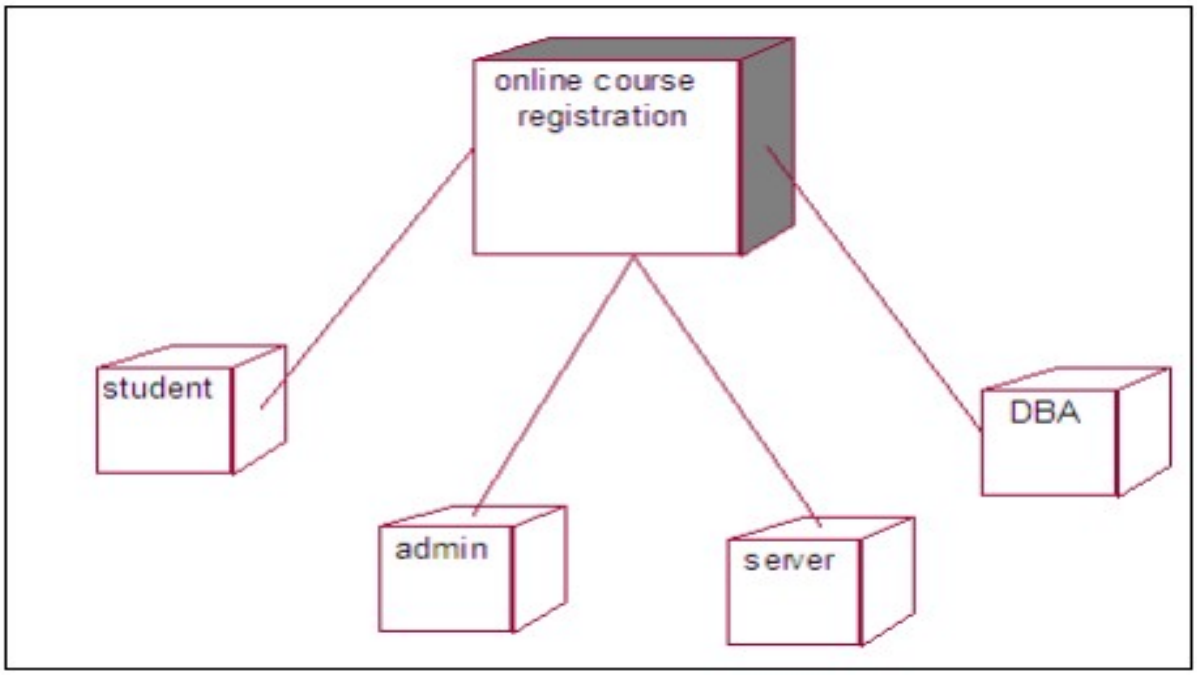
ATM



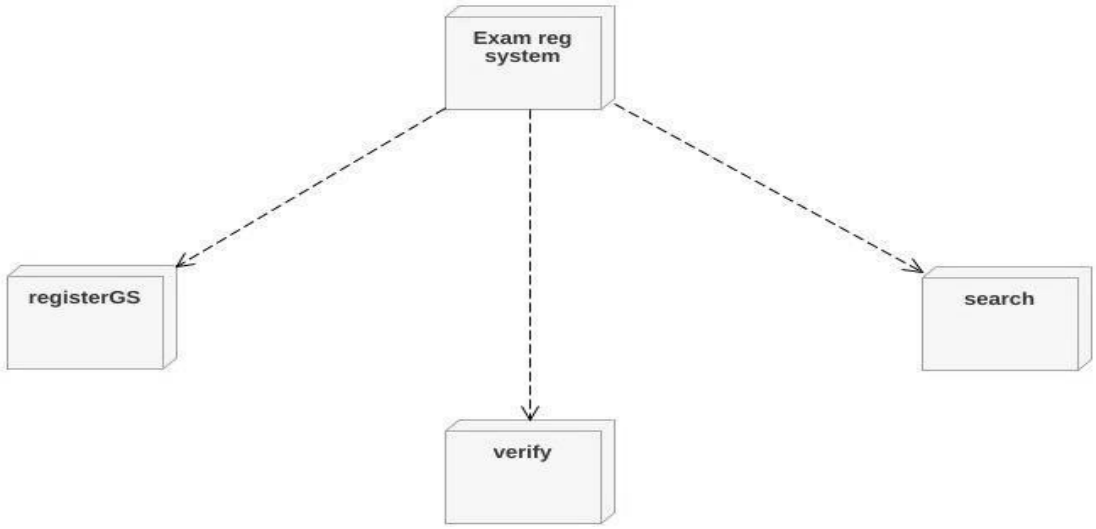
Deployment Diagram for Online Shopping



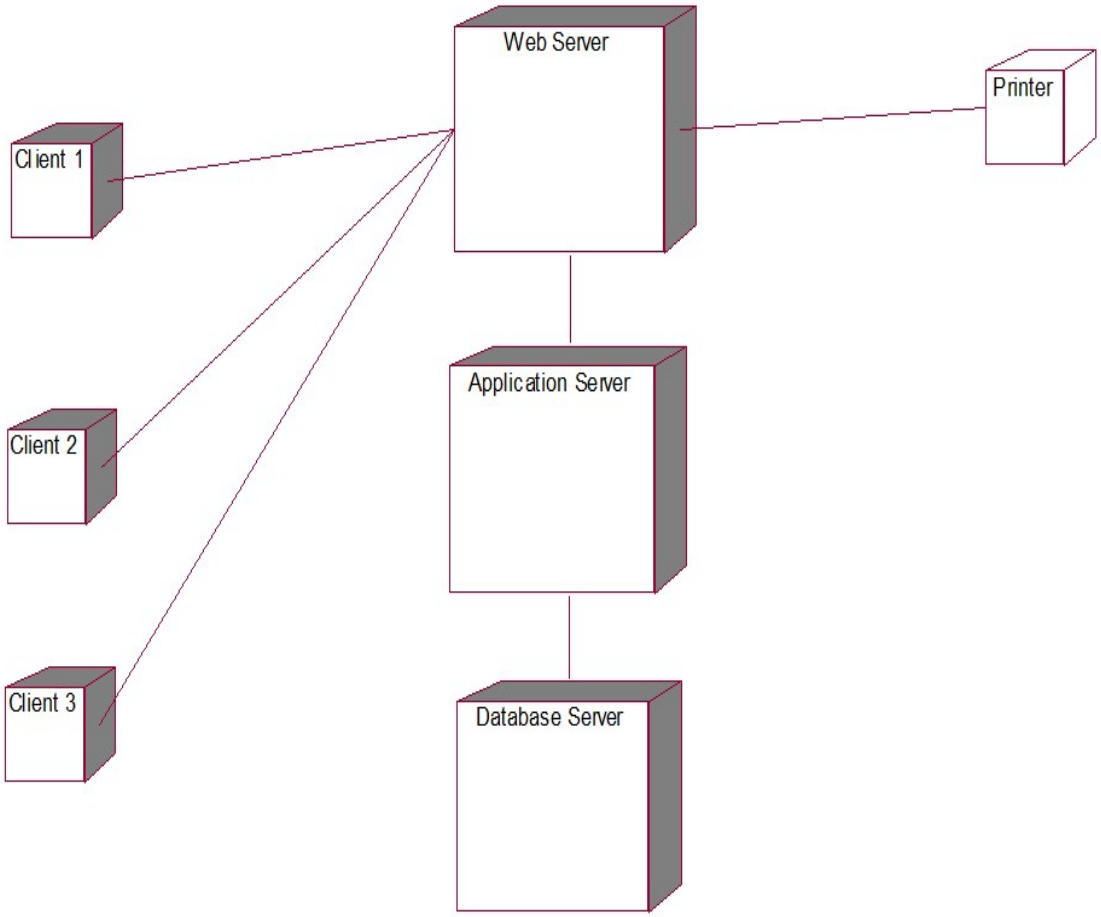
Course Registration



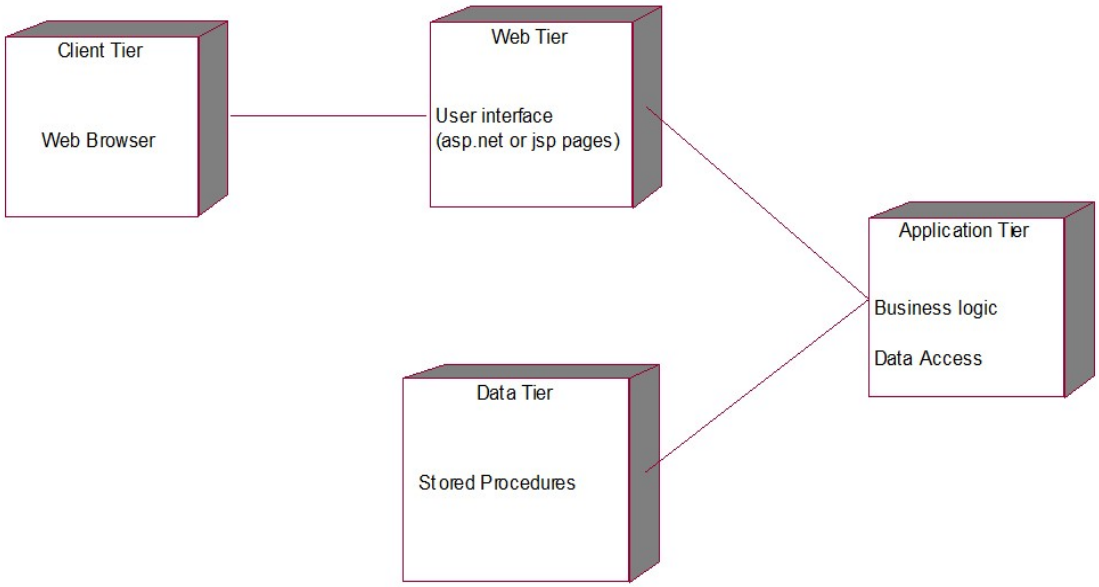
Exam Registration



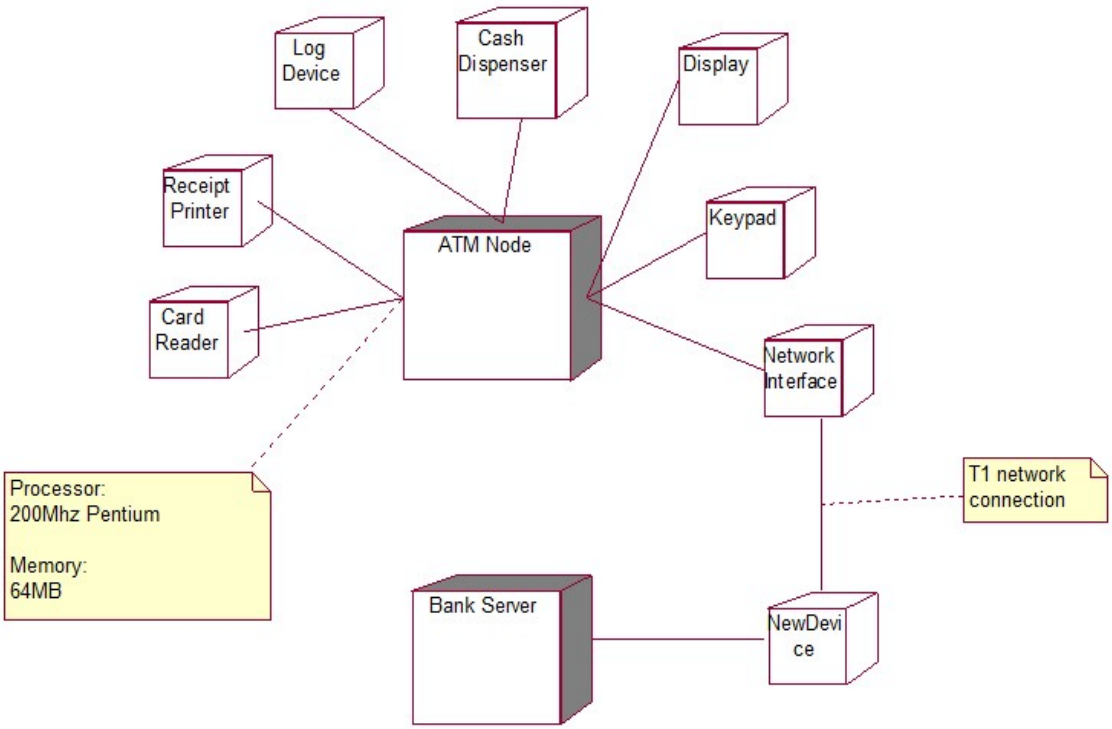
Railway Reservation



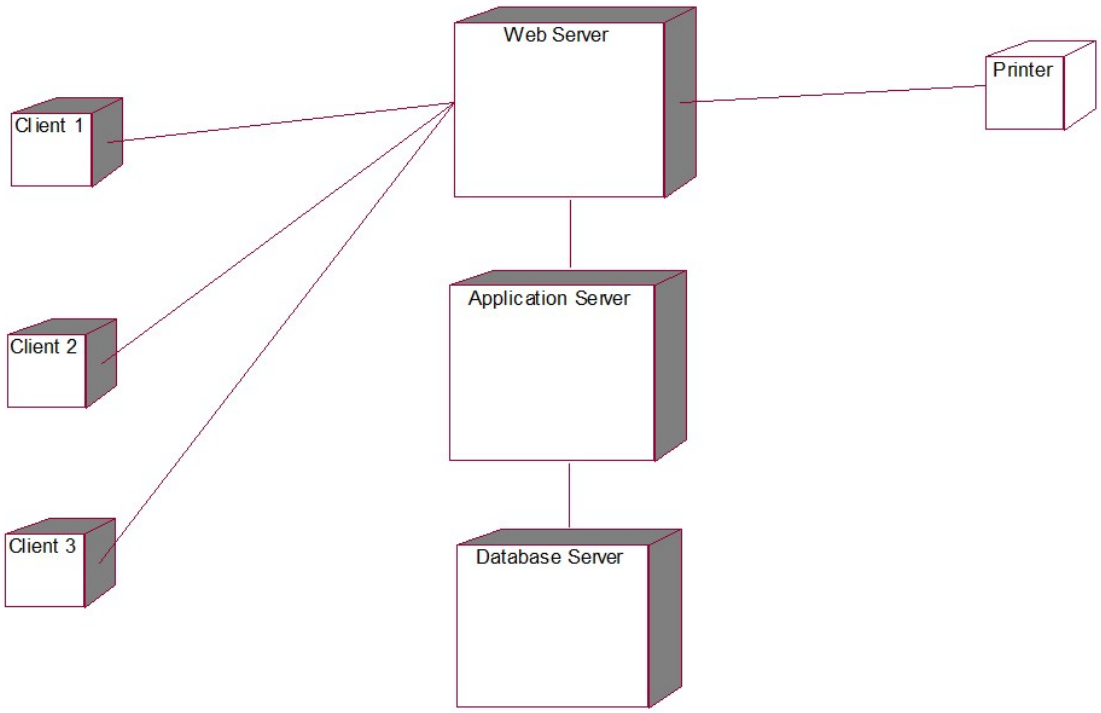
Library Management Systems



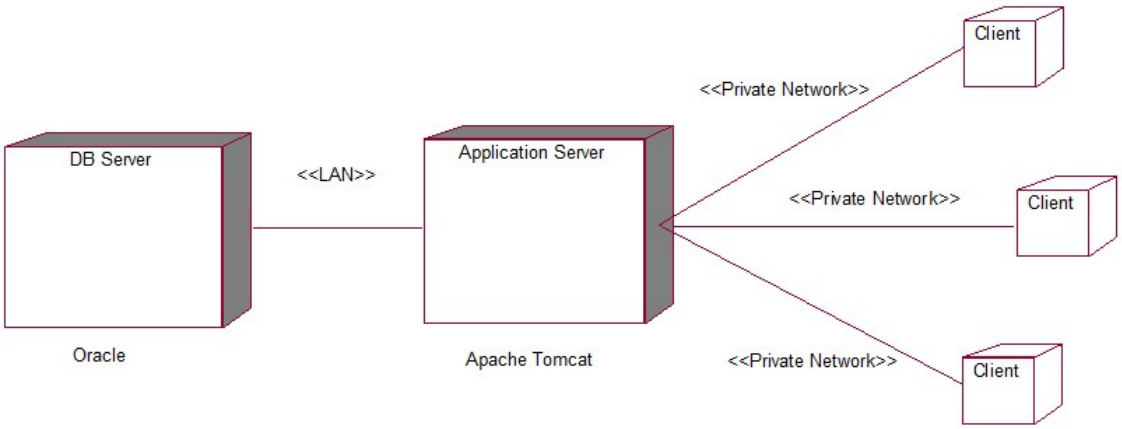
ATM



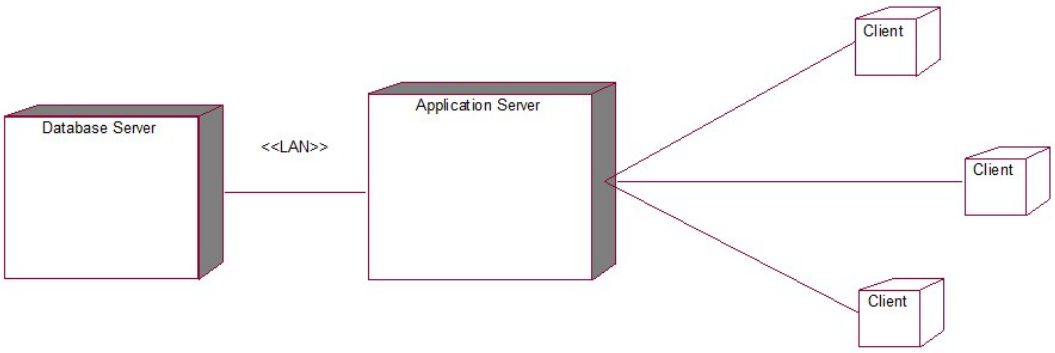
Railway Reservation



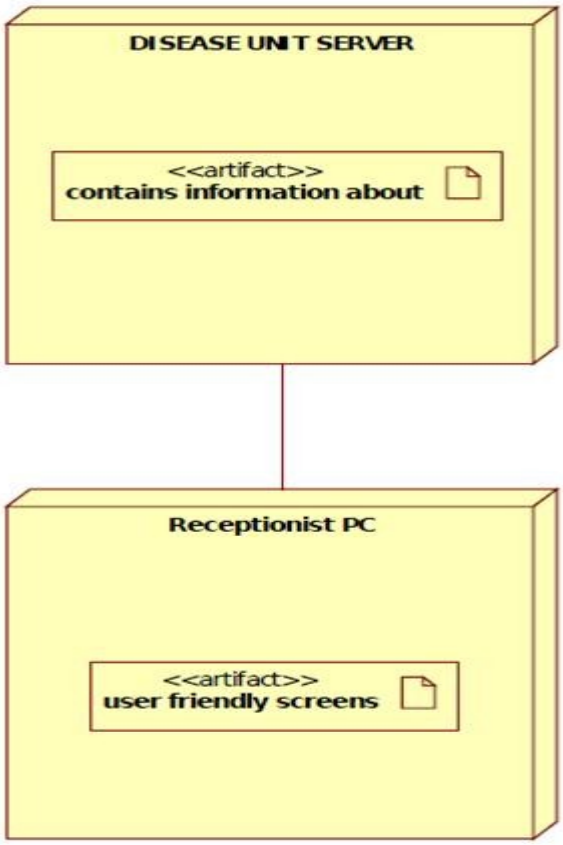
Online Book Shopping



Online Banking



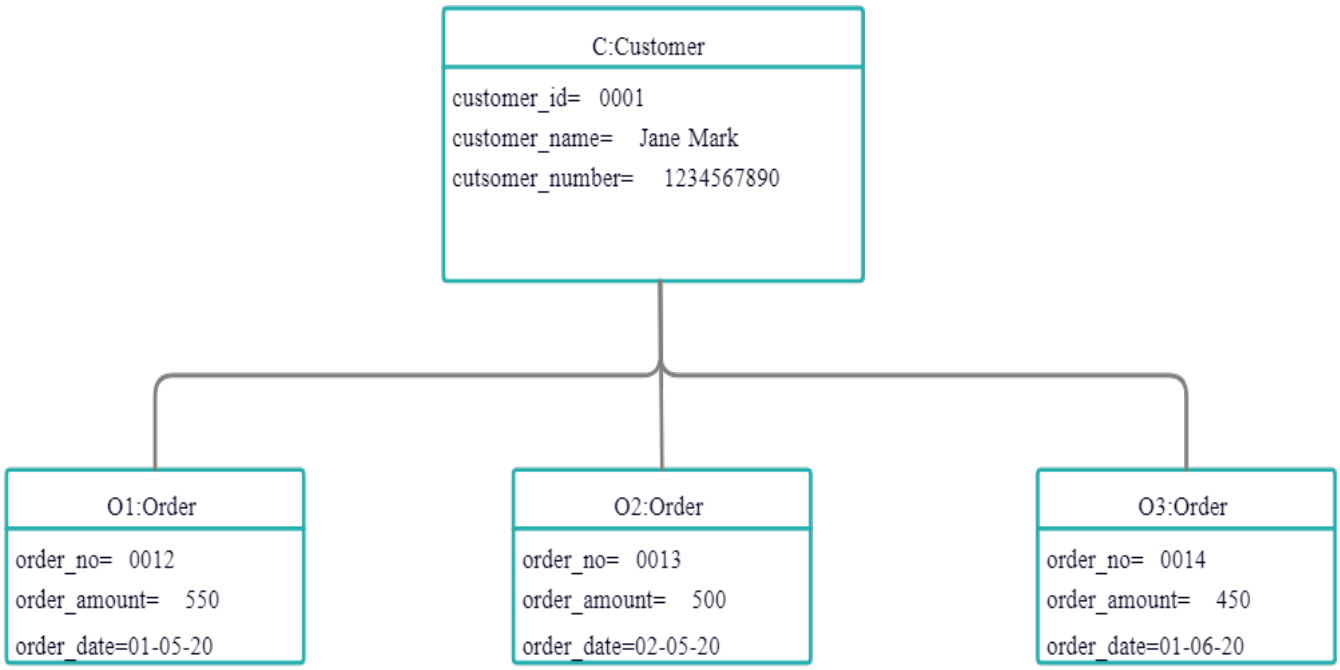
HMS

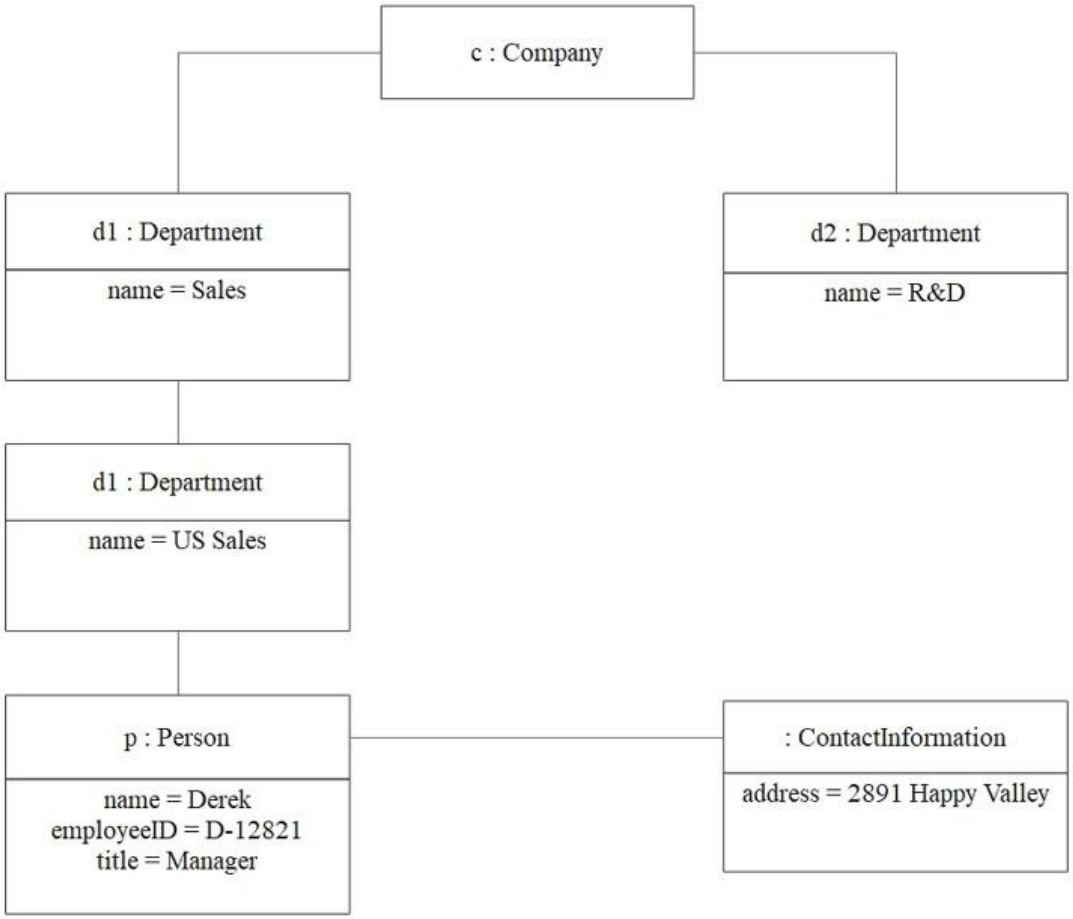


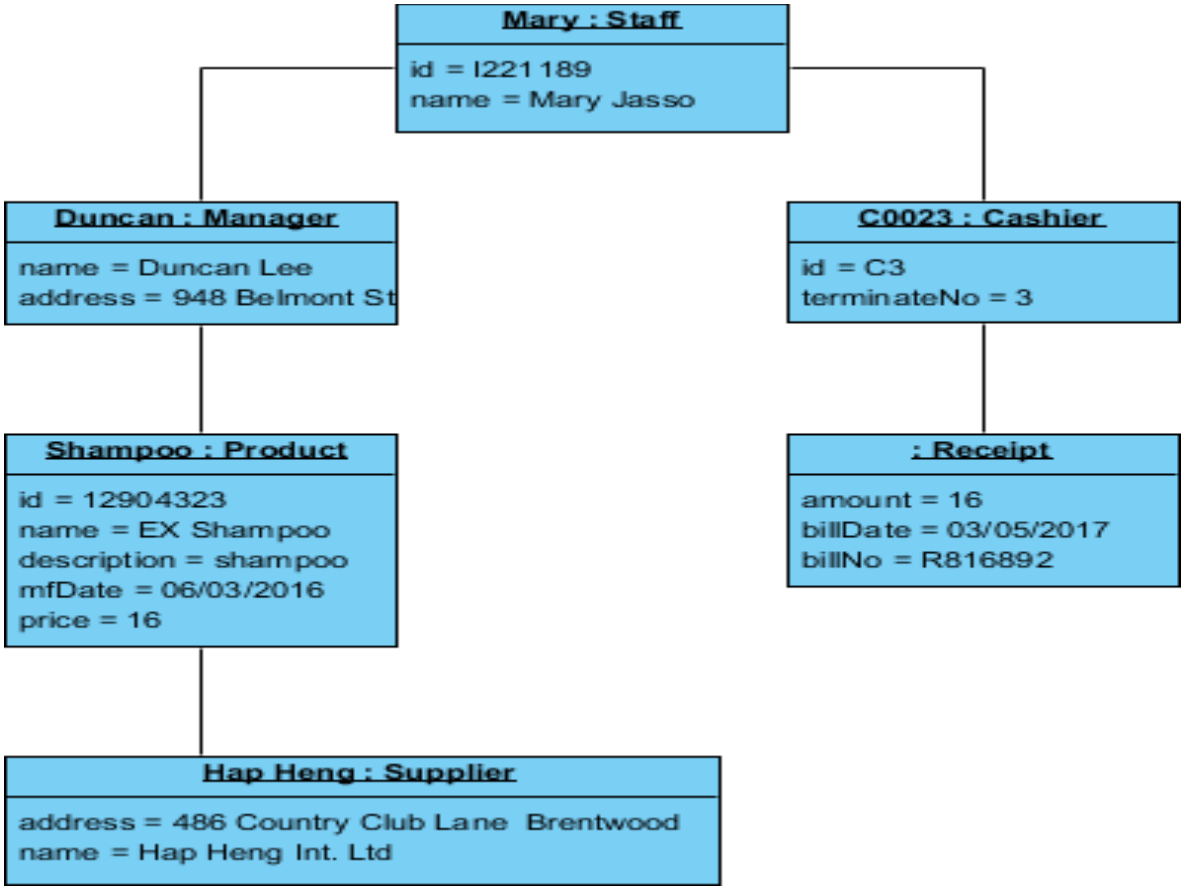
Object Diagram

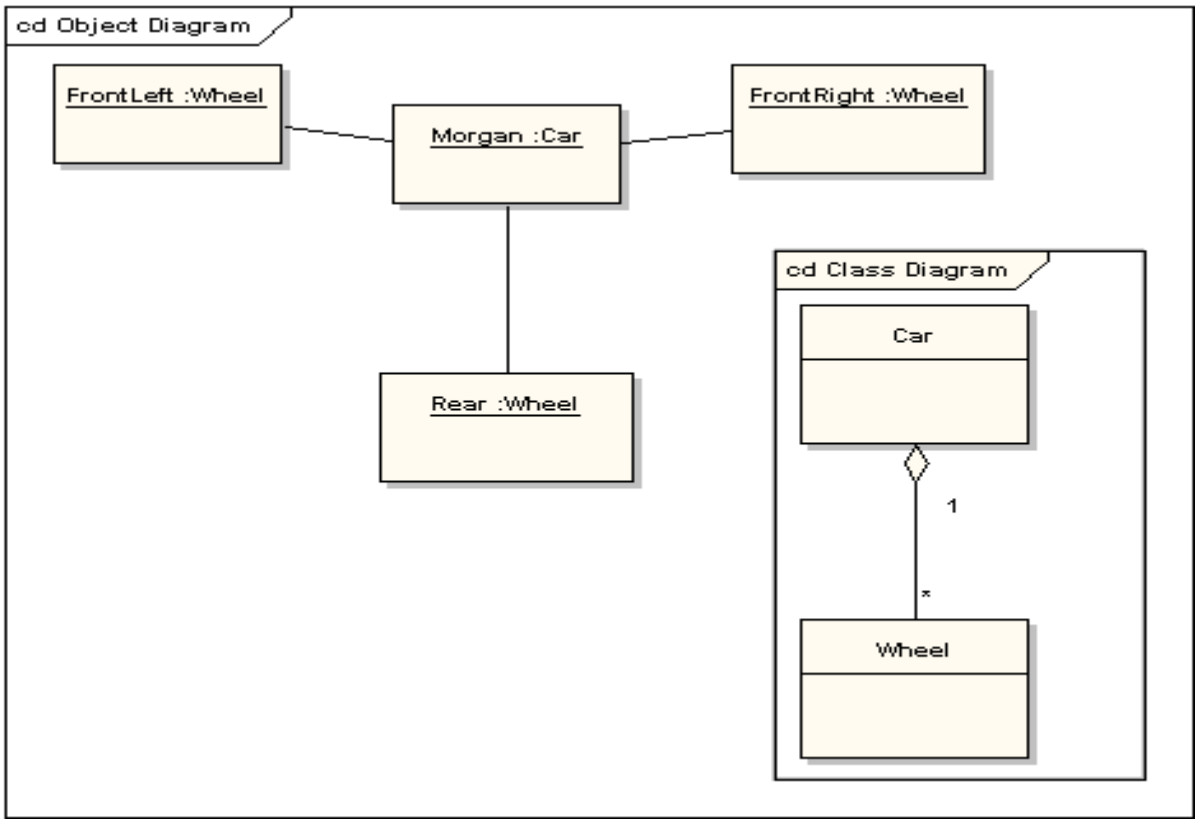
An object diagram in Unified Modelling Language (UML) is a structural diagram that shows a set of objects and their relationships at a particular point in time within a system. It provides a snapshot of the instances of classes in a system and how they collaborate and interact with each other.

UML Object diagrams are very closely related to class diagrams. While both diagrams afford the user a static, bird's eye view of a system, object diagrams represent a particular instance of a class diagram at a given point in time. They represent the functionality of a class, and how objects within a class relate to each other.




c : Company
d1 : Department
name = Sales
d2 : Department
name = R&D
d1 : Department
name = US Sales
p : Person
name = Derek
employeeID = D-12821
title = Manager
: ContactInformation
address = 2891 Happy Valley


Mary : Staff
id = I221189
name = Mary Jasso
Duncan : Manager
name = Duncan Lee
address = 948 Belmont St
C0023 : Cashier
id = C3
terminateNo = 3
Shampoo : Product
id = 12904323
name = EX Shampoo
description = shampoo
mfDate = 06/03/2016
price = 16
: Receipt
amount = 16
billDate = 03/05/2017
billNo = R816892
Hap Heng : Supplier
address = 486 Country Club Lane Brentwood
name = Hap Heng Int. Ltd


cd Object Diagram
FrontLeft :Wheel
Morgan :Car
FrontRight :Wheel
Rear :Wheel
cd Class Diagram
Car
1
*
Wheel

Programs On Exception Hadling In Pl/Sql

1. Zero Divide Exception

```
DECLARE
   v_num1 NUMBER := 100;
   v_num2 NUMBER := 0;
   v_result NUMBER;
BEGIN
   v_result := v_num1 / v_num2;
   DBMS_OUTPUT.PUT_LINE('Result: ' || v_result);
EXCEPTION
   WHEN ZERO_DIVIDE THEN
      DBMS_OUTPUT.PUT_LINE('Error: Cannot divide a number by
zero.');
END;
/
```

2. No Data Found Exception

```
DECLARE
   v_emp_name VARCHAR2(50);
BEGIN
   -- Searching for an employee ID that does not exist
   SELECT first_name INTO v_emp_name
   FROM employees
   WHERE employee_id = 9999;

   DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_emp_name);
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Error: No record found for the given
Employee ID.');
END;
/
```

3. Too Many Rows Exception

```
DECLARE
   v_emp_name VARCHAR2(50);
BEGIN
   -- Query returns multiple rows because department_id isn't unique
   SELECT first_name INTO v_emp_name
   FROM employees
```

```
    WHERE department_id = 10;

    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_emp_name);
EXCEPTION
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Error: Query returned more than one
row. Use a cursor instead.');
END;
/
```

4. User-Defined Exception

```
DECLARE
    v_age NUMBER := 15;
    e_underage EXCEPTION; -- Declare custom exception
BEGIN
    IF v_age < 18 THEN
        RAISE e_underage; -- Trigger custom exception
    END IF;

    DBMS_OUTPUT.PUT_LINE('Access granted. Welcome!');
EXCEPTION
    WHEN e_underage THEN
        DBMS_OUTPUT.PUT_LINE('Error: User must be at least 18 years
old.');
END;
/
```